

CENTRO UNIVERSITÁRIO FEI
LEONARDO APARECIDO DA SILVA

**Desenvolvimento de um Painel de Controle e Comando para o gerenciamento de Robôs
Humanoides**

São Bernardo do Campo

2025

LEONARDO APARECIDO DA SILVA

**Desenvolvimento de um Painel de Controle e Comando para o gerenciamento de Robôs
Humanoides**

Relatório Parcial de Iniciação Científica
apresentado ao Centro Universitário FEI, como
parte dos requisitos do Programa PIBIC-FEI.
Orientado pelo Prof. Reinaldo Augusto da C
Bianchi.

São Bernardo do Campo

2025

RESUMO

Este relatório parcial de iniciação científica apresenta o desenvolvimento de um painel de controle e comando integrado para o gerenciamento de robôs humanoides da equipe RoboFEI. O trabalho tem como objetivo centralizar informações de status e automatizar processos de inicialização e diagnóstico, visando reduzir o tempo de manutenção e a dependência de infraestruturas externas como monitores e periféricos. A metodologia emprega uma abordagem iterativa que compreende o levantamento de dados quantitativos sobre o processo atual, o projeto conceitual de hardware baseado no microcontrolador ESP32-S3 e a integração com o middleware ROS 2 Humble Hawksbill. Os resultados preliminares, obtidos por meio de questionários de percepção e medições objetivas, revelam que a montagem física e a inicialização manual consomem, em média, 289 segundos, evidenciando uma disparidade de desempenho de até 87% entre membros de diferentes subequipes. A análise técnica do protótipo indica um potencial de redução de 85% no tempo total de inicialização e de até 97% no diagnóstico de falhas críticas de hardware, como sensores IMU e câmeras. Conclui-se que a implementação da interface física proposta promove a universalização do uso do robô e otimiza a eficiência operacional em ambientes de competição.

Palavras-chave: Robôs humanoides; Painéis de controle; Sistemas embarcados; ROS 2; Interface homem-máquina.

LISTA DE ILUSTRAÇÕES

Figura 1 – WABOT-1.....	9
Figura 2 – Comparação entre sistemas de malha aberta e malha fechada.....	12
Figura 3 –KUKA SmartPAD-2.....	15
Figura 4 – Grafo de nós e tópicos do sistema ROS gerado com o rqt_graph.....	24
Figura 5 – Layout da placa de potência do sistema.....	32
Figura 6 – Layout da placa de comunicação do sistema.....	33
Figura 7 – Infraestrutura necessária para operação do robô no método atual, ilustrando a necessidade de múltiplos periféricos e espaço físico adequado.....	36

LISTA DE TABELAS

Tabela 1 – Comparação técnica entre protocolos UART e USB	21
Tabela 2 – Tempo estimado para ativar a decisão do robô após ligar o NUC.....	43
Tabela 3 – Tempo para identificar se a Câmera está funcionando.....	44
Tabela 4 – Tempo para identificar se a Placa de Comunicação está funcionando.....	45
Tabela 5 – Tempo para identificar se a IMU está funcionando.....	46
Tabela 6 – Tempo para identificar se um motor tem overload.....	47
Tabela 7 – Tempo para identificar se o robô está conectado ao Game Controller.....	48
Tabela 8 – Tempo estimado para demonstração no cenário atual.....	49
Tabela 9 – Tempo estimado para demonstração com painel de controle	50
Tabela 10 – Análise do tempo empregado para tarefas básicas em segundos.....	51
Tabela 11 – Análise do tempo empregado para identificar falhas em segundos.....	52
Tabela 12 – Comparativo: Expectativa da Equipe vs. Medições Reais em segundos....	54
Tabela 13 – Funcionalidades mais mencionadas na pesquisa de campo.....	58
Tabela 14 – Comandos da interface de entrada (Botões).....	59
Tabela 15 – Comandos da interface de saída (LEDS).....	60
Tabela 16 – Análise comparativa de etapas: processo atual vs. processo com painel....	61
Tabela 17 – Comparação de tempo de diagnóstico de falhas.....	64

SUMÁRIO

1	INTRODUÇÃO	6
2	REVISÃO BIBLIOGRÁFICA	8
2.1	ROBÔS.....	8
2.2	SISTEMAS DE CONTROLE	11
2.3	PAINÉIS DE CONTROLE	13
2.3.1	Painéis de controle na robótica	15
2.4	SISTEMAS EMBARCADOS E MICROCONTROLADORES	17
2.5	PROTOCOLOS DE COMUNICAÇÃO	19
2.5.1	Protocolo UART	21
2.5.2	Protocolo USB.....	23
2.6	ROS2	24
2.7	ARQUITETURA DE SOFTWARE DO ROBÔ.....	26
2.8	SISTEMAS ELÉTRICOS DO ROBÔ	31
3	METODOLOGIA	37
3.1	MOTIVAÇÃO E PROBLEMÁTICA	37
3.2	MATERIAIS UTILIZADOS.....	39
3.3	METODOLOGIA DE DESENVOLVIMENTO.....	41
3.3.1	Levantamento de Dados sobre o Processo Atual	41
3.3.2	Projeto Conceitual e Definição de Requisitos	42
3.3.3	Desenvolvimento de Protótipo Funcional.....	43
3.3.4	Coleta de Dados Preliminares com Protótipo	44
4	RESULTADOS E DISCUSSÃO	45
5	CRONOGRAMA.....	69
6	CONCLUSÕES	70
	REFERÊNCIAS	72

1 INTRODUÇÃO

A equipe RoboFEI Humanoide, do Centro Universitário FEI, é um projeto institucional que, desde 2003, promove o estudo e o desenvolvimento de robôs humanoides capazes de participar de partidas de futebol de forma autônoma. O projeto abriga um ambiente de pesquisa contínua, no qual inovações tecnológicas são regularmente integradas aos robôs, resultando em avanços constantes nas capacidades de percepção, controle e movimentação.

À medida que esses sistemas se tornam mais complexos, cresce o número de potenciais pontos de falha e o esforço requerido para diagnóstico e manutenção. A identificação de problemas passa a demandar mais tempo de inspeção e verificação de múltiplos subsistemas, o que impacta diretamente a eficiência das atividades de desenvolvimento. Em ambientes de competições, como a CBR, essa limitação se torna ainda mais relevante, uma vez que o tempo disponível para ajustes é reduzido e cada segundo pode influenciar o desempenho da equipe no campeonato.

Outro desafio recorrente refere-se à operação dos robôs por um número reduzido de integrantes da equipe. Em situações nas quais apenas uma pessoa está presente, torna-se necessário manter o robô fisicamente estável, isto é, mantê-lo em pé enquanto são realizados procedimentos de configuração, o que é agravado pela necessidade de conectar e utilizar simultaneamente múltiplos periféricos, como teclado, mouse, monitor e cabos HDMI. Essa combinação de fatores aumenta o tempo de preparação, dificulta o uso do robô em locais sem infraestrutura adequada e favorece a ocorrência de falhas que podem permanecer indetectáveis por longos períodos, comprometendo a confiabilidade do sistema e reduzindo o tempo disponível para treinamento. A implementação de uma interface dedicada, composta por botões e indicadores visuais, apresenta potencial para agilizar a interação com o robô, simplificar o processo de configuração, reduzir a necessidade de apoio de outros integrantes da equipe e minimizar o impacto de falhas decorrentes de manutenções incompletas.

Diante desse contexto, este trabalho propõe o desenvolvimento de um painel de controle integrado para o robô humanoide, com o objetivo de centralizar informações de status do sistema, monitorar conexões de sensores e fornecer indicações visuais do

funcionamento de nós de software que demandam maior atenção ou não apresentam feedback direto. Essa solução tem potencial para reduzir significativamente o tempo de diagnóstico e manutenção, simplificar o envio de comandos e aumentar a disponibilidade do robô para treinamentos e competições. Este projeto contribui para o avanço das ferramentas de monitoramento em robôs humanoides ao propor uma solução de baixo custo e facilmente replicável.

2 REVISÃO BIBLIOGRÁFICA

2.1 ROBÔS

De acordo com a norma ISO 8373:2021 (INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2021), um robô é definido como “um mecanismo atuador programado que realiza locomoção, manipulação ou posicionamento”. Essa definição, por ser bem técnica, reflete precisamente a visão contemporânea da robótica, sendo amplamente aceita pela comunidade científica e pela engenharia. No entanto, essa concepção não representa a visão inicial acerca do tema, cuja origem é fortemente marcada por aspectos literários e filosóficos. Antes mesmo de processos formais de industrialização se quer existirem, civilizações antigas, como a egípcia e a grega helenística, já idealizavam mecanismos que sugeriam a ideia de robôs.

Em meados de 1495, o famoso pintor Leonardo da Vinci, por exemplo, a partir de seu minucioso conhecimento de anatomia, desenvolveu um “cavaleiro mecânico”. De acordo com o Institute and Museum of the History of Science (2002):

Este autômato humanoide vestia uma armadura típica do design germano-italiano do século XV, semelhante à usada pela milícia milanese. O cavaleiro mecânico era capaz de sentar-se, mover os braços e girar a cabeça através de um pescoço articulado, além de abrir e fechar a mandíbula anatomicamente correta. Ele emitia sons acompanhados por tambores automatizados. Estudos em andamento no Istituto e Museo di Storia della Scienza, em Florença, sugerem que o robô possuía dois sistemas mecânicos independentes: o primeiro com pernas, tornozelos, joelhos e quadris com três graus de liberdade, e braços com ombros, cotovelos, punhos e mãos com quatro graus de liberdade. (Institute and Museum of the History of Science, 2002).

Logo, é notável que os conceitos de automação e inteligência mecânica antecedem o processo de industrialização. Ainda que não existisse qualquer tipo de eletrônica ou linguagem de programação, a capacidade de imaginar e a destreza na engenharia permitiram que essa invenção fosse concretizada e, futuramente, aprimorada.

O termo “robô” deriva das palavras tchecas *rabota* (“trabalho”) e *robotnik* (“servo”), sendo popularizado em 1921 na peça teatral *R.U.R. – Rossum’s Universal Robots*, escrita pelo dramaturgo Karel Čapek. A obra introduziu a ideia de seres artificiais criados para substituir o trabalho humano, abordando questões de alienação e mecanização. Esse imaginário de servidão e multifuncionalidade consolidou a base conceitual que moldaria a percepção pública e científica dos robôs nas décadas seguintes (ČAPEK, 1921; KATO, 2005).

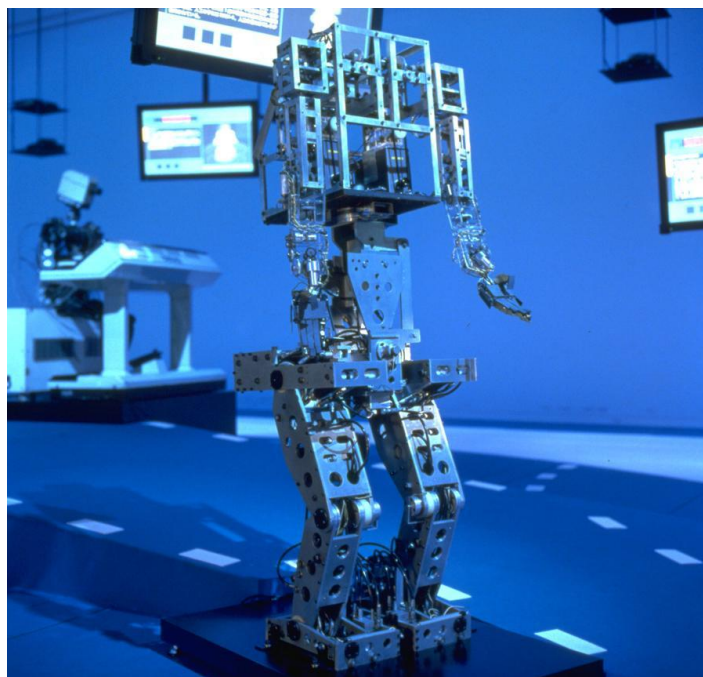
Com o avanço da industrialização e a busca pela modernização dos processos produtivos, especialmente após as revoluções industriais, emergiu a necessidade de máquinas capazes de executar tarefas repetitivas, precisas e potencialmente perigosas. Nesse contexto surgiu o Unimate, amplamente reconhecido como o primeiro robô industrial. Desenvolvido na década de 1950 por George Devol, o Unimate consistia em um braço mecânico programável: “[...] o primeiro Unimate foi instalado na fábrica da General Motors localizada em Trenton (EUA); ele podia realizar uma única tarefa, especificamente extrair peças de uma máquina de fundição sob pressão” (GASPARETTO; SCALERA, 2018, p. 3). Essa inovação inaugurou a era da robótica industrial moderna, redefinindo paradigmas de produtividade e segurança no ambiente fabril.

Nas décadas seguintes, os robôs industriais tornaram-se elementos centrais em processos que exigem alta repetibilidade, precisão e resistência a condições extremas. Paralelamente, a robótica expandiu-se para além do ambiente fabril, dando origem a novas categorias: os robôs de serviço, voltados a tarefas domésticas ou logísticas; os robôs agrícolas, utilizados para manejo automatizado de lavouras; e, sobretudo, os robôs humanoides, concebidos para interagir com pessoas em contextos educacionais, culturais e recreativos.

O desenvolvimento dos robôs humanoides representa uma transição crucial na história da robótica, unindo aspectos mecânicos, eletrônicos e cognitivos. Um marco significativo nesse campo foi o projeto WABOT-1, desenvolvido na década de 1970 pelo Waseda University Humanoid Robotics Institute: “O WABOT-1 foi o primeiro robô antropomórfico em escala real desenvolvido no mundo” (KATO, 2005; WASEDA UNIVERSITY, 2010). Segundo a universidade, o robô humanoide podia se comunicar

em japonês. A partir de iniciativas como o WABOT, a robótica humanoide passou a se consolidar como uma área de pesquisa interdisciplinar, voltada à reprodução de capacidades humanas em sistemas artificiais.

Figura 1 – WABOT-1



Fonte: Referência [26]

No Brasil, grupos de pesquisa como a RoboFEI, vinculada ao Centro Universitário FEI, têm contribuído ativamente para o avanço da robótica. A equipe é referência nacional e internacional no desenvolvimento de diferentes categorias, como robôs humanoides, robôs de serviço e industriais e robôs small-sized. Isso demonstra o quanto o campo da robótica se expandiu, pois atualmente é possível utilizar robôs para realizar tarefas domésticas, como aspiradores de pó, ou para interagir com ambientes inteligentes e executar diversas funções. Além disso, robôs têm forte presença cultural e de lazer, sendo capazes de reproduzir partidas de futebol, dançar e interagir com pessoas. Cada vez mais autônomos e integrados a sistemas de Internet das Coisas (IoT) e inteligência artificial, eles podem aprender e se adaptar a novos cenários por meio de machine learning. Esse avanço, antes restrito à indústria, hoje é impulsionado também por estudantes e pesquisadores de graduação, o que representa um ponto fundamental para o desenvolvimento tecnológico da nova geração.

Em síntese, a trajetória dos robôs, desde sua concepção literária até sua materialização como sistemas inteligentes e autônomos, reflete o contínuo entrelaçamento entre ciência, tecnologia e sociedade. A evolução da robótica, impulsionada por inovações industriais e acadêmicas, culmina hoje em um campo interdisciplinar que integra engenharia, computação e cognição artificial, delineando o futuro da interação entre humanos e máquinas.

2.2 SISTEMAS DE CONTROLE

A previsibilidade de resultados é um fator determinante para a aceitação e a confiabilidade de processos, sejam eles industriais ou naturais. Quanto maior a previsibilidade do desfecho, mais segura e eficiente torna-se a operação. Diante da necessidade de gerenciar sistemas para obter resultados específicos, desenvolveu-se a engenharia de controle, fundamental para o avanço tecnológico e a qualidade de vida moderna.

Segundo a norma IEC 60050-351:2013, um sistema de controle pode ser definido como um conjunto de elementos interconectados que formam uma estrutura capaz de atingir um objetivo determinado. Tais sistemas são onipresentes, variando desde mecanismos biológicos complexos, como o sistema respiratório humano, que regula trocas gasosas, até aplicações industriais, como um painel de controle que aciona um motor elétrico. Neste último, o botão atua como entrada de referência e o acionamento do motor como a saída desejada. A aplicação correta desses sistemas permite automatizar tarefas com precisão e velocidade superiores às capacidades humanas, impactando diretamente o cotidiano e a indústria.

Historicamente, o uso de mecanismos de controle precede a Revolução Industrial. Nise (2012, p. 31) aponta que "os gregos começaram a engenharia de sistemas com realimentação por volta de 300 a.C. [...] usando-se uma válvula de boia semelhante à do controle de nível de água da caixa de descarga dos vasos sanitários atuais". Entretanto, foi com a demanda por produção em massa e eficiência energética que a teoria se consolidou. Sobre a evolução desses mecanismos, Nise (2012, p. 32) descreve:

A regulação da pressão do vapor começou por volta de 1681, com a invenção da válvula de segurança por Denis Papin. O conceito foi aprimorado aumentando-se o peso do topo da válvula. Se a pressão ascendente oriunda da caldeira excedesse o peso, o vapor era liberado, e a pressão diminuía. [...] Também no século XVII, Cornelis Drebbel, na Holanda, inventou um sistema de controle de temperatura puramente mecânico para a incubação de ovos.

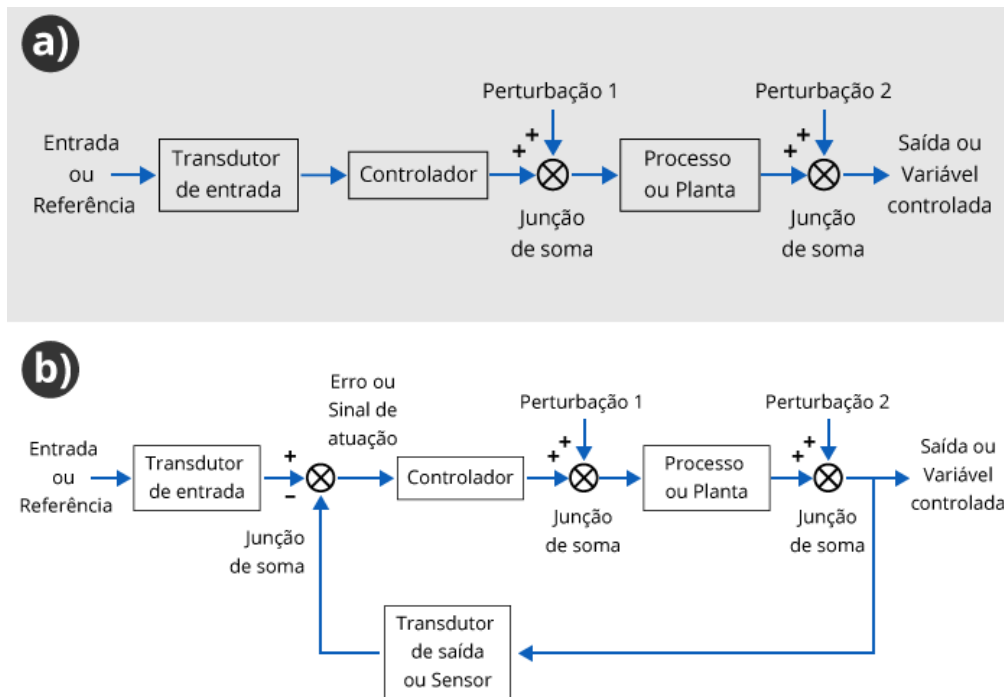
A teoria de controle moderna começou a se sedimentar na segunda metade do século XIX (NISE, 2012). Atualmente, suas aplicações são vastas, especialmente na robótica e na engenharia biomédica. Drones e mísseis utilizam controle automático para geolocalização e estabilização. Na área da saúde, busca-se mimetizar sistemas biológicos, como o pâncreas artificial. Anteriormente, pacientes diabéticos dependiam de medições manuais invasivas e injeções de insulina, um processo doloroso e sujeito a falhas. Hoje, sistemas de controle em malha fechada com sensores contínuos monitoram a glicemia e administram insulina automaticamente, simulando a função do órgão e melhorando a qualidade de vida do paciente.

Em síntese, um sistema de controle opera manipulando uma variável de entrada para obter uma resposta desejada na saída. Existem duas configurações fundamentais: malha aberta e malha fechada. Segundo Nise (2012), os sistemas de malha aberta são aqueles que não possuem mecanismos para monitorar ou corrigir sua própria saída; ou seja, não compensam perturbações externas. Um exemplo simples é uma máquina de lavar roupa, que opera por tempo pré-determinado sem verificar se a roupa está efetivamente limpa.

Por outro lado, os sistemas de malha fechada, ou sistemas com realimentação, caracterizam-se pela capacidade de comparar a resposta real da saída com a entrada desejada e utilizar a diferença (erro) para corrigir o processo (NISE, 2012). Essa realimentação confere ao sistema capacidade de autorregulação e maior complexidade, permitindo a correção de erros gerados por perturbações ou variações nos parâmetros do sistema.", isso confere ao sistema capacidade de autorregulação e maior complexidade. Um exemplo avançado ocorre na robótica humanoide, como na competição RoboCup. Robôs da equipe CIT-Brains, por exemplo, utilizam sistemas de malha fechada com sensores extensômetros (*strain-gauges*) nos pés e unidades de medição inercial (IMU). O controlador processa os dados de equilíbrio em tempo real e ajusta os atuadores para

evitar quedas, corrigindo o erro entre a posição atual e a desejada, algo impossível de ser realizado com precisão em um sistema de malha aberta.

Figura 2 – Comparação entre sistemas de malha aberta e malha fechada



Fonte: Referência [15]

Finalmente, conclui-se que os sistemas de controle são imprescindíveis para garantir a estabilidade e a eficiência de processos, sejam eles de malha aberta ou fechada. Tais sistemas asseguram a previsibilidade e o gerenciamento de resultados em diversas áreas, desde aplicações industriais até tecnologias aplicadas à saúde. Neste contexto, a complexidade de sistemas como robôs humanoides demanda uma supervisão eficaz. Para tanto, a utilização de um painel de controle torna-se essencial, permitindo o envio de comandos de alto nível e garantindo uma operação segura, fundamentada nos princípios da teoria de controle.

2.3 PAINÉIS DE CONTROLE

Dada a necessidade de centralizar o comando e o monitoramento de máquinas, computadores e sistemas complexos, a engenharia desenvolveu o conceito de painéis de controle. De maneira geral, um painel de controle pode ser entendido como uma interface física ou digital que permite acompanhar o funcionamento de um sistema de forma

progressiva e interagir com ele por meio de comandos específicos. Embora esse conceito seja amplo, diferentes áreas da engenharia atribuem significados próprios ao termo, conforme sua aplicação.

No âmbito computacional, a interação entre usuário e computador é frequentemente mediada por Interfaces Gráficas do Usuário (GUI). Segundo a norma ISO 9241-161:2016, interfaces gráficas utilizam metáforas visuais e elementos de controle (widgets) para permitir a manipulação direta de objetos digitais. Nesse contexto, “painel de controle” refere-se a um utilitário de software que agrupa configurações do sistema operacional, sem correspondente físico, destinado ao gerenciamento de parâmetros e recursos lógicos.

Já na engenharia elétrica, por sua vez, o painel assume um caráter físico de infraestrutura. Segundo a norma ABNT NBR IEC 60204-1, que regula equipamentos elétricos de máquinas, o dispositivo caracteriza-se como um Equipamento de Controle, alocado em um invólucro específico para proteção e operação. De forma mais ampla, a norma IEC 61439-1 define esta montagem como um CONJUNTO de manobra e comando, integrando dispositivos de proteção e distribuição de potência interconectados elétrica e mecanicamente. No Brasil, tais definições alinham-se à Norma Regulamentadora NR-12, que estabelece requisitos rigorosos para "Dispositivos de partida, acionamento e parada", exigindo que os painéis possuam interfaces claras, seguras e que impeçam acionamentos acidentais.

Contudo, para a operação efetiva de um robô, não basta apenas a distribuição elétrica ou a configuração de software; é necessária uma Interface Homem-Máquina (IHM). Segundo Groover (2011, p. 122), "o painel de controle do operador é a interface básica pela qual o operador da máquina se comunica com o sistema", servindo como ponte vital para a troca de informações. Tecnicamente, conforme descreve Bolton (2010), essa comunicação ocorre através de transdutores: sensores de contato (botões e chaves) convertem a ação física do operador em sinais elétricos de entrada, enquanto indicadores (LEDs) atuam como transdutores de saída, convertendo sinais do sistema em informação visual.

Historicamente, as IHMs passaram por uma evolução significativa. Inicialmente, os painéis eram estritamente "cabeados" (hardwired), compostos por vastos arranjos de relés, botões físicos e lâmpadas piloto, onde cada função exigia um componente dedicado. Com o avanço da eletrônica e o surgimento dos Controladores Lógicos Programáveis (CLP), houve uma migração para interfaces baseadas em telas e displays digitais, permitindo maior flexibilidade e densidade de informação, conforme discutido nas diretrizes da norma ISA 101 sobre eficiência visual em interfaces.

Apesar da modernização e do uso crescente de telas sensíveis ao toque, interfaces físicas mantêm sua relevância em aplicações críticas devido à robustez e à ergonomia. Sob a ótica de fatores humanos, Sanders e McCormick (1993, p. 14) definem um sistema homem-máquina como "uma combinação de um ou mais seres humanos e um ou mais componentes físicos interagindo para produzir, a partir de entradas dadas, alguma saída desejada". Em situações de controle em tempo real, a retroalimentação tátil (o "clique" físico de um botão) oferece uma confirmação sensorial que telas sensíveis ao toque muitas vezes não conseguem replicar com a mesma eficácia e segurança.

Neste projeto, optou-se por uma abordagem híbrida que valoriza essa interação física. O painel desenvolvido atua como o componente tátil que viabiliza a comunicação imediata, permitindo que o operador envie comandos de segurança e operação através de botões físicos, obtendo como resposta o comportamento desejado do robô humanoide. Simultaneamente, o dispositivo desempenha a função de monitoramento de status, utilizando sinalização luminosa (LEDs) para alertar o operador sobre falhas elétricas ou de software, garantindo uma supervisão contínua.

2.3.1 Painéis de controle na robótica

De maneira geral, robôs são empregados em diversas áreas para realizar tarefas que atendam às demandas humanas. Eles estão presentes na agricultura, indústrias, comércios e até mesmo em ambientes domésticos, muitas vezes desempenhando ações repetitivas ou perigosas.

Tendo em vista a necessidade de implementar a interação humano-máquina, faz-se necessária a utilização de painéis de controle nesses robôs. Isso se justifica tanto pela

segurança, pois tais dispositivos portam botões de emergência que interrompem a operação em situações de risco, conforme indicado pelas normas NR-12 e ISO 10218, quanto pelo gerenciamento da operação, uma vez que exibem o comportamento progressivo do robô, oferecendo maior ergonomia e conforto aos usuários.

No contexto industrial, o dispositivo padrão para esta função é denominado Teach Pendant (Pendente de Ensino). Conforme descreve Groover (2011), trata-se de um console portátil provido de interruptores, teclas e mostradores digitais, utilizado para guiar o robô através de sequências de movimento e armazenar essas trajetórias na memória do controlador.

O smartPAD, da empresa alemã KUKA, destaca-se, a título de exemplo, como um painel de controle utilizado nos robôs industriais da marca. Conforme as especificações técnicas do fabricante, o dispositivo atua como um Teach Pendant para o manipulador, sendo equipado com tablet, estrutura de proteção (shells), dispositivo de parada de emergência e cabos de conexão (KUKA DEUTSCHLAND GMBH, 2021, p. 7).

Figura 3 –KUKA SmartPAD-2



Fonte: Referência [24]

Outrossim, é possível observar a utilização de painéis em robôs desenvolvidos em universidades destinados a competições, como a RoboCup. A equipe TEAM DYNAMICS, por exemplo, utiliza uma unidade de controle que “facilita a operação do robô, na qual diversos switches foram instalados para ligar/desligar funções até mesmo semiautônomas” (EDLINGER, 2022, p. 6, tradução nossa). Já a equipe ITAndroids possui a Rear Interface Board (RIB), uma interface equipada com botões para facilitar a

entrada de comandos, contendo infraestrutura de programação e LEDs para diagnóstico (JUSTA et al., 2019, p. 4, tradução nossa). Esses exemplos revelam que os painéis permitem um controle mais refinado e diagnósticos rápidos durante as competições.

Ademais, observa-se que painéis são amplamente utilizados para a teleoperação de robôs, contexto no qual a interface cognitiva desempenha um papel crucial. Segundo Zheng et al. (2025):

A interface cognitiva entre o robô e o operador humano é de suma importância no supracitado processo de manufatura robótica teleoperada, pois ajuda a facilitar a comunicação eficaz, a colaboração e a troca de informações entre o operador humano e o robô, o que oferece ao operador humano uma maneira mais intuitiva de controlar o robô remotamente, melhorando, em última análise, o desempenho da manufatura (ZHENG et al., 2025, p. 4283, tradução nossa).

Por fim, conclui-se que a utilização de painéis na robótica é versátil e de extrema importância para dinamizar a operação. Eles proporcionam mais segurança, ergonomia e conforto para os usuários, pois possibilitam integrar uma interação humano-máquina eficiente e responsiva.

2.4 SISTEMAS EMBARCADOS E MICROCONTROLADORES

O controle de interfaces físicas e a automação de processos necessitam essencialmente de uma unidade de processamento capaz de interpretar sinais e gerar comandos de saída em tempo real. Esses dispositivos, caracterizados por realizar funções dedicadas dentro de um sistema maior, são denominados sistemas embarcados. Diferenciando-os de computadores de uso geral, Almeida, Valério e Moura (2016, p. 1) definem que "sistemas embarcados são sistemas eletrônicos microprocessados que, após serem programados, possuem uma função específica que geralmente não pode ser alterada".

Essa especificidade funcional permite que o *hardware* seja otimizado para a tarefa, priorizando baixo consumo de energia, tamanho reduzido e robustez operacional.

O componente central dessa arquitetura é a unidade de processamento. Nesse ponto, Bolton (2010) estabelece uma distinção fundamental entre microprocessadores e microcontroladores. O microprocessador refere-se apenas à Unidade Central de Processamento (CPU), exigindo componentes externos para se tornar funcional, como memória e interfaces de entrada e saída.

O microcontrolador, por sua vez, pode ser compreendido como um sistema computacional completo integrado em um único circuito integrado. Segundo Godse e Godse (2009), esse dispositivo reúne, em uma única pastilha de silício, a unidade central de processamento (CPU), a memória de programa destinada ao armazenamento do firmware, a memória de dados e as portas de entrada e saída (I/O). Tal integração confere aos microcontroladores características ideais para aplicações de controle, uma vez que elimina a necessidade de barramentos externos complexos e contribui para a redução do custo final do projeto.

Do ponto de vista estrutural, Godse e Godse (2009) descrevem que um microcontrolador típico é constituído pelos seguintes blocos funcionais interconectados: a CPU, responsável pela busca e execução de instruções; a memória, segmentada em ROM/Flash, de carácter não volátil, destinada ao armazenamento do código, e RAM, de natureza volátil, utilizada para variáveis temporárias; além dos periféricos de entrada e saída, que viabilizam a comunicação com dispositivos externos.

No cenário tecnológico atual, diversas famílias de componentes ilustram essa arquitetura. O Arduino Mega 2560, amplamente utilizado em prototipagem, baseia-se no microcontrolador de 8 bits ATmega2560, oferecendo 54 pinos de entrada/saída digital e múltiplas interfaces seriais (UART). Para aplicações que exigem maior desempenho, a linha STM32F103 emprega núcleos ARM Cortex-M3 de 32 bits operando a até 72 MHz, integrando interfaces nativas USB e CAN em um único encapsulamento. Avançando para sistemas conectados, o SoC (*System on Chip*) ESP32-S3 exemplifica a evolução dessa tecnologia ao integrar processadores de núcleo duplo (*Dual-core*) Xtensa® LX7 com conectividade Wi-Fi e Bluetooth, mantendo 45 GPIOs programáveis para interação com periféricos.

Para o desenvolvimento do painel de controle deste projeto, a característica mais relevante do microcontrolador é a presença dessas portas de Entrada/Saída de Propósito Geral (GPIO). Segundo Zanco (2005), as portas de I/O são os canais pelos quais o dispositivo lê o estado de sensores e controla atuadores. No contexto da aplicação, o microcontrolador atua como o elemento de inteligência local, sendo responsável por realizar a varredura constante (*polling*) do estado dos botões físicos e gerenciar o acionamento dos LEDs de *status*, preparando essas informações para serem transmitidas ao robô via protocolo serial.

2.5 PROTOCOLOS DE COMUNICAÇÃO

A evolução de sistemas robóticos e computacionais esta diretamente ligada ao desenvolvimento de meios eficientes de comunicação entre dispositivos. De acordo com a norma ISO/IEC 7498-1 (1994), o Modelo de Referência para Interconexão de Sistemas Abertos (OSI) foi estabelecido para fornecer uma base comum que viabilizasse a coordenação e o desenvolvimento de padrões para a interconexão de sistemas. A norma define uma arquitetura abstrata dividida em sete camadas hierárquicas, onde cada nível desempenha funções específicas de gerenciamento, transporte e processamento de dados. Essa estrutura modular assegura que sistemas computacionais heterogêneos, independentemente de seus fabricantes ou arquiteturas internas, possam comunicar-se entre si, desde que sigam os protocolos padronizados de interfaces e serviços definidos para cada camada.

Em sistemas embarcados e robóticos, diversos protocolos podem ser empregados para comunicação entre módulos, tais como I²C, SPI, CAN, UART e Ethernet. Cada um apresenta compromissos distintos entre taxa de dados, complexidade de implementação, número de dispositivos suportados e robustez a ruídos. A escolha do protocolo adequado depende diretamente dos requisitos da aplicação, como volume de dados, latência, confiabilidade e custo do sistema.

Para que o painel de controle atue efetivamente como uma interface de entrada e saída, é necessário estabelecer um meio robusto de troca de dados entre o hardware (painel) e o controlador principal do robô. Segundo Bolton (2010), a interconexão de dispositivos inteligentes requer não apenas uma conexão física, mas também um acordo

prévio sobre as regras de troca de dados, denominado protocolo, responsável por organizar o formato das mensagens, o sincronismo entre transmissor e receptor, os mecanismos de detecção de erros e o controle da velocidade de transmissão. Conforme explica o autor, todo protocolo se fundamenta em três elementos essenciais: a sintaxe, que determina a estrutura e codificação dos dados, a semântica, que rege o controle de fluxo, a sinalização e o tratamento de erros e a temporização, que estabelece a ordem e o ritmo com que as informações são trocadas.

Além desses aspectos, a comunicação pode ser classificada segundo a direção e simultaneidade do fluxo de dados. No modo simplex, as informações trafegam em um único sentido. No modo half-duplex, a transmissão é bidirecional, porém não simultânea, exigindo que cada bloco enviado seja reconhecido antes da continuação da troca de mensagens. Já o modo full-duplex permite envio e recepção simultâneos, proporcionando maior eficiência quando há necessidade de comunicação constante entre duas unidades.

Em sistemas sujeitos a ruído, como robôs móveis operando com cabos longos, motores, inversores e atuadores de alta potência, a integridade dos dados transmitidos é um aspecto crítico. Bolton (2010) destaca que protocolos frequentemente incorporam métodos de detecção de erros, como a Verificação de Redundância Cíclica (CRC). Nesse método, o transmissor realiza uma operação matemática sobre os dados e envia o resultado junto com a mensagem, permitindo que o receptor repita o cálculo e compare os valores, garantindo que o conteúdo não tenha sido corrompido durante o tráfego.

Com base nesses princípios, observa-se que a definição correta do protocolo é essencial para assegurar que o painel de controle e o robô realizem uma troca de informações confiável, estável e resiliente a ruídos. Isso é particularmente importante em aplicações robóticas, nas quais decisões incorretas provocadas por falhas na comunicação podem comprometer a segurança, o desempenho ou até a integridade física da máquina. Dentre as diversas opções disponíveis para a interface entre microcontroladores e computadores centrais, destacam-se os protocolos UART e USB.

2.5.1 Protocolo UART

A comunicação serial assíncrona é um dos métodos mais fundamentais para a troca de dados em sistemas embarcados. Para implementar esse protocolo via hardware, utiliza-se o periférico denominado UART (Universal Asynchronous Receiver/Transmitter). Segundo Bolton (2010), a UART é o circuito responsável por converter os dados paralelos do barramento interno do microcontrolador em um fluxo serial de bits para transmissão, e vice-versa na recepção.

A característica distintiva deste protocolo é a ausência de um sinal de clock compartilhado para sincronizar o emissor e o receptor. Em vez disso, a sincronização é estabelecida através de bits de controle inseridos no pacote de dados. Um quadro (frame) típico de transmissão UART inicia-se com um start bit (nível lógico baixo), seguido pelos bits de dados (geralmente 8 bits), um bit opcional de paridade para verificação de erros e, finalmente, um ou mais stop bits (nível lógico alto) que indicam o fim do pacote.

Para que a comunicação ocorra com sucesso, ambos os dispositivos devem ser configurados previamente com a mesma taxa de transmissão, conhecida como baud rate. No contexto do painel de controle, a UART atua como o estágio primário de comunicação: o microcontrolador lê o estado dos botões e formata essas informações em pacotes seriais, transmitindo-os através dos pinos TX (transmissão) e RX (recepção) disponíveis no hardware.

Em aplicações robóticas acadêmicas e industriais, a UART é amplamente utilizada para comunicação entre microcontroladores e computadores embarcados, especialmente em sistemas que empregam middleware como o ROS. Nesses casos, os dados trafegam através de portas seriais virtuais, normalmente identificadas como “/dev/ttyUSBx” em sistemas Linux. Apesar da interface física ser USB, os dados transmitidos continuam obedecendo ao protocolo UART. Essa abordagem é comum em plataformas baseadas em Arduino e ESP32, nas quais a simplicidade do protocolo e a baixa sobrecarga computacional favorecem a confiabilidade da comunicação.

Na robótica, a UART é frequentemente a "linguagem nativa" de muitos atuadores inteligentes e sensores, servindo como base para padrões industriais como o RS-232 e o RS-485. Sua simplicidade de *hardware* permite que microcontroladores processem

comandos com baixíssima sobrecarga computacional, sendo ideal para malhas de controle de tempo real.

Para melhor situar as diferenças entre a comunicação lógica (UART) e a interface física (USB) abordada na sequência, apresenta-se um comparativo técnico. A Tabela 1 destaca as principais características e aplicabilidades de cada protocolo no contexto de sistemas embarcados robóticos.

Tabela 1 - Comparação técnica entre protocolos UART e USB

Característica	UART	USB
Tipo de comunicação	Serial assíncrona ponto-a-ponto	Serial síncrona host-device
Sincronização	Bits de controle (start/stop)	Clock compartilhado (gerenciado pelo host)
Taxa de transmissão típica	9.600 a 115.200 bps (configurável até ~1 Mbps)	1,5 Mbps (USB 1.0) a 480 Mbps (USB 2.0)
Distância máxima recomendada	~15 metros (RS-232) / ~1.200 metros (RS-485)	5 metros (USB 2.0) / 3 metros (USB 3.0)
Número de fios de dados	2 (TX e RX) + GND	4 (D+, D-, VCC e GND)
Alimentação elétrica	Não fornece	Fornece (5V, até 500mA em USB 2.0)

Tabela 1 - Comparação técnica entre protocolos UART e USB

Característica	UART	USB
Complexidade de implementação	Baixa (hardware simples)	Média (requer controlador USB ou conversor)
Robustez a ruídos	Média (sinais single-ended)	Alta (sinais diferenciais)
Plug-and-play	Não (configuração manual de baud rate)	Sim (autoconfiguração pelo SO)
Custo relativo	Muito baixo	Baixo a médio
Aplicação no painel	Comunicação lógica entre microcontrolador e conversor	Conexão física com computador do robô

Fonte: Adaptado de Bolton (2010).

2.5.2 Protocolo USB

Enquanto a UART resolve a comunicação lógica nível-a-nível entre chips, a conexão física com computadores modernos exige interfaces mais robustas e padronizadas. O USB (Universal Serial Bus) consolidou-se como o padrão industrial para conexão de periféricos devido à sua capacidade de autoconfiguração (plug-and-play) e altas taxas de transferência.

Conforme descreve Bolton (2010), o USB opera sob uma arquitetura mestre-escravo (host-device), onde o computador (host) gerencia todo o tráfego de dados. Fisicamente, utiliza par trançado para transmissão diferencial de sinais (D+ e D-), o que

confere maior imunidade a ruídos eletromagnéticos em comparação à comunicação serial simples.

No desenvolvimento de interfaces robóticas, é comum a utilização de conversores USB-Serial integrados às placas de desenvolvimento. Componentes como o ESP32-S3 possuem suporte nativo ou interfaces dedicadas que encapsulam os dados da UART dentro do protocolo USB. Isso permite que, ao conectar o painel de controle ao computador do robô, o sistema operacional reconheça o dispositivo automaticamente como uma "Porta Serial Virtual" (VCP). Essa característica reduz significativamente a complexidade de integração, permitindo que softwares de alto nível, como o ROS2, comuniquem-se com o hardware embarcado de forma transparente (acessando caminhos como `/dev/ttyUSBx`), sem a necessidade de drivers proprietários complexos.

Em arquiteturas de robôs humanoides complexos, onde múltiplos dispositivos (câmeras, placas de controle de motores, sensores LiDAR) precisam ser conectados a um computador central, o USB oferece a vantagem da padronização. Através de *hubs* e identificadores únicos de dispositivo (*Vendor ID* e *Product ID*), o sistema operacional do robô consegue mapear e gerenciar cada periférico de forma independente, facilitando a manutenção e a escalabilidade do sistema.

2.6 ROS2

O desenvolvimento de robôs móveis autônomos, que integram percepção computacional, controle de movimento e tomada de decisão, exige uma arquitetura de software robusta, capaz de gerenciar múltiplos processos simultâneos e heterogêneos. Historicamente, pesquisadores despendiam grande parte do tempo de projeto criando drivers e protocolos proprietários, o que dificultava o compartilhamento de soluções. Para mitigar esse problema, surgiu o ROS (Robot Operating System). Apesar da nomenclatura, Quigley, Gerkey e Smart (2015) esclarecem que o ROS não é um sistema operacional no sentido tradicional (como Windows ou Linux), mas sim um middleware robótico: uma camada de software estruturada que fornece abstração de hardware, drivers de dispositivos, bibliotecas de comunicação e ferramentas de visualização, operando sobre o sistema operacional do computador hospedeiro.

A filosofia central do ROS baseia-se na distribuição e modularização de processos. Joseph (2018) define essa estrutura através do conceito de Nós (Nodes), que são unidades de execução independentes, onde cada processo é responsável por uma tarefa específica, como ler um sensor ou controlar a velocidade de uma roda. Para que esses nós operem de forma coordenada, o sistema utiliza canais de comunicação denominados Tópicos (Topics), que funcionam como barramentos de dados assíncronos. Nesse contexto, a interação ocorre através de dois agentes fundamentais: o Publicador (Publisher), nó responsável por gerar e enviar dados para um tópico, e o Assinante (Subscriber), nó que se inscreve no tópico para consumir essas informações. Essa arquitetura garante o desacoplamento do sistema, pois publicadores e assinantes não precisam conhecer a existência uns dos outros para interagir. Para organizar esses elementos, Quigley, Gerkey e Smart (2015) introduzem o conceito de Pacotes (Packages). O pacote é a unidade atômica de construção e distribuição de software no ROS, agrupando nós, bibliotecas, datasets e arquivos de configuração em uma estrutura padronizada, o que facilita a compilação, a reutilização e o compartilhamento de código entre diferentes projetos.

Além da comunicação assíncrona via tópicos, o framework oferece Serviços (Services) para trocas síncronas de solicitação-resposta e Ações (Actions) para tarefas de longa duração. Embora a primeira versão do ROS tenha se tornado o padrão de facto na robótica acadêmica, ela apresentava limitações para aplicações de tempo real devido à dependência de um servidor central de gerenciamento (Master). Para superar essas barreiras, foi lançado o ROS 2. A principal inovação desta nova versão é a adoção do padrão industrial DDS (Data Distribution Service) como camada de transporte. Téllez, Ezquerro e Rodríguez (2019) explicam que o uso do DDS elimina a necessidade de um mestre centralizado, permitindo que a descoberta de nós seja dinâmica e descentralizada, o que aumenta significativamente a confiabilidade, a segurança e o suporte a sistemas de tempo real. Essa evolução foi crucial para viabilizar a integração direta de microcontroladores ao ecossistema ROS (via Micro-ROS) e para suportar arquiteturas distribuídas complexas.

Essa transição tecnológica reflete-se diretamente na trajetória da equipe RoboFEI. A análise dos Team Description Papers (TDPs) demonstra uma progressão clara: de arquiteturas baseadas em códigos monolíticos e frameworks específicos nas competições

anteriores a 2017, para a adoção completa do paradigma modular nos anos recentes. Atualmente, o robô opera sobre a versão ROS 2 Humble Hawksbill, dividindo suas funções vitais, como visão, decisão, controle de movimento e comunicação em pacotes independentes, conforme detalhado por Saleme et al. (2025). Neste cenário, o painel de controle desenvolvido neste projeto atua como um periférico de hardware integrado à rede ROS: o microcontrolador envia os dados físicos via porta serial/USB e um nó de interface no computador do robô (atuando como Publisher) converte esses sinais em mensagens padronizadas nos tópicos de comando, permitindo que a estratégia de jogo (atuando como Subscriber) reaja aos inputs do operador de forma transparente.

Em suma, o ROS consolida-se não apenas como uma ferramenta de desenvolvimento, mas como o elemento integrador fundamental entre o hardware (sensores e atuadores), o firmware (microcontroladores) e o software de alto nível (inteligência artificial e controle). Sua adoção neste projeto é essencial para garantir que o painel de controle seja compatível com os padrões da robótica moderna. Além disso, o domínio dessa tecnologia é uma competência crítica para a formação dos futuros engenheiros, visto que o framework promove o aprendizado de conceitos complexos de sistemas distribuídos, reutilização de código e colaboração em larga escala, preparando os estudantes para os desafios da Indústria 4.0.

2.7 ARQUITETURA DE SOFTWARE DO ROBÔ

A arquitetura de software do robô humanoide foi desenvolvida sobre o middleware Robot Operating System 2 (ROS 2), especificamente na distribuição Humble Hawksbill, operando sobre o sistema operacional Ubuntu Linux. O processamento computacional é centralizado em uma unidade Intel NUC, responsável por executar a lógica de controle e tomada de decisão. Esta plataforma foi selecionada por oferecer desempenho compatível com o processamento em tempo real, requisito fundamental para a operação autônoma em competições robóticas. A integração entre o software e o hardware físico do robô é viabilizada pelo sistema eletroeletrônico, que não apenas fornece energia aos componentes através do painel de distribuição, mas também estabelece todas as conexões de comunicação necessárias entre sensores, atuadores e a unidade de processamento central.

O sistema foi projetado para atender rigorosamente às diretrizes estabelecidas no documento "Laws of the Game 2025" da RoboCup Humanoid League. Na categoria KidSize, as partidas ocorrem em um campo de aproximadamente 9 metros de comprimento por 6 metros de largura, disputadas por duas equipes de até quatro robôs cada. Conforme as regras de "Comunicação e Controle", os robôs devem atuar com autonomia total, sem qualquer intervenção humana, controle remoto ou processamento externo ("remote brain") durante o tempo regulamentar. Para cumprir este requisito de autonomia, o software deve gerenciar não apenas a estratégia de jogo, mas fundamentalmente a estabilidade da marcha bípede e a recuperação autônoma em caso de quedas, uma capacidade mandatória para a categoria KidSize.

A implementação do código segue os princípios de modularidade do ROS 2, estruturada em pacotes independentes que se comunicam através de tópicos (topics), serviços (services) e ações (actions). O código é escrito majoritariamente em C++ (cerca de 95% do código-fonte) para garantir a eficiência computacional em operações críticas de tempo real, complementado por scripts auxiliares em Python. O fluxo de dados prioriza a reatividade e a estabilidade, integrando subsistemas que operam de forma concorrente e assíncrona. O código-fonte completo está disponível no repositório GitHub da equipe.

O subsistema de Visão (vision_pkg) constitui o principal nó de percepção do sistema, processando as imagens capturadas pela câmera montada na cabeça do robô. Este módulo respeita as restrições impostas pela regra de "Sensores", que exige sensores equivalentes aos sentidos humanos, posicionados na cabeça do robô e com campo de visão limitado a 180 graus. O algoritmo YOLOv8 (You Only Look Once) é utilizado para a detecção de objetos em tempo real, identificando elementos cruciais como a bola, as traves dos gols e outros robôs. Atualizações recentes otimizaram a classificação espacial da bola através do uso de duas linhas verticais e duas horizontais como referências visuais, permitindo ao robô estimar a posição relativa da bola sem a necessidade de um mapa global complexo. O nó de visão publica continuamente as coordenadas detectadas em tópicos ROS 2 específicos, disponibilizando essas informações para os demais módulos do sistema.

A Unidade de Medição Inercial (IMU), gerenciada pelo nó um7, desempenha um papel central na arquitetura, sendo vital para a estabilidade dinâmica. Dados de acelerômetros e giroscópios são processados em alta frequência (tipicamente 100-200 Hz)

para monitorar continuamente o equilíbrio e a orientação do robô. Este sensor é indispensável para cumprir a regra que exige que robôs da KidSize se levantem sozinhos após uma queda: ao detectar uma inclinação crítica através dos valores de roll (inclinação lateral) e pitch (inclinação frontal), o sistema aciona automaticamente os protocolos de levante (get-up motions). Além disso, os dados inerciais alimentam o controle de malha fechada da caminhada, permitindo ajustes em tempo real para manter a postura ereta, conforme exigido pela definição de design dos robôs. O nó um7 publica os dados da IMU em tópicos padrão do ROS 2, que são subscritos tanto pelo módulo de controle quanto pelo módulo de decisão.

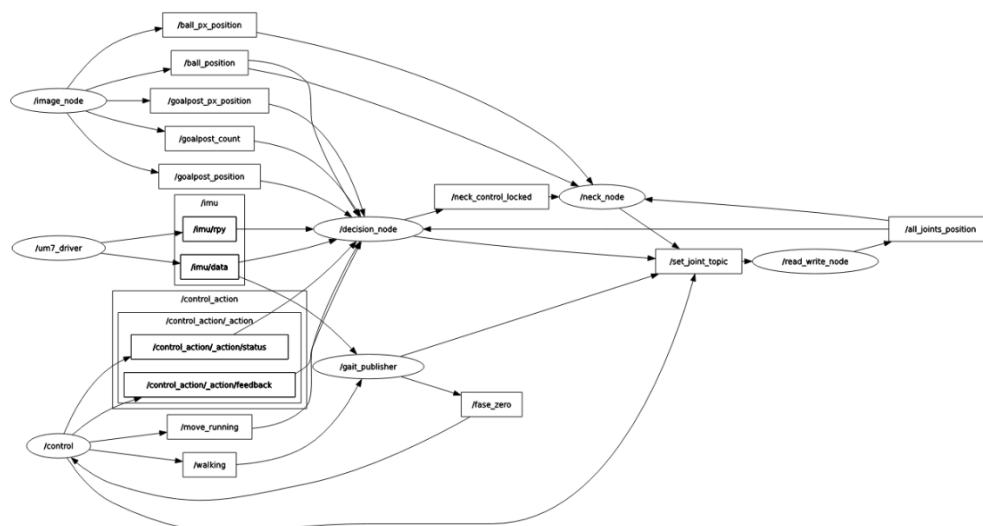
No nível cognitivo, o nó de Decisão (decision_pkg) implementa a inteligência artificial e as estratégias de jogo. Este pacote comunica-se com o Game Controller oficial da liga, um software externo que transmite os estados da partida (como Ready, Set, Playing) e informações sobre penalidades via protocolo UDP. O nó de decisão subscrive os tópicos publicados pelos módulos de visão e IMU, processando essas informações para determinar o comportamento apropriado do robô a cada instante. Com base nos dados sensoriais e no estado do jogo, o robô define sua ação imediata, que pode incluir comportamentos como procurar a bola, aproximar-se dela, alinhar-se com o gol adversário ou executar um chute. A lógica atual implementa duas estratégias principais: a primeira, mais agressiva, prioriza uma abordagem de "andar e chutar" (walk and shoot), onde o robô se orienta ativamente em direção à bola para realizar o chute ao gol adversário; a segunda, mais conservadora, adota uma estratégia de "empurrar", onde o robô conduz a bola através de uma caminhada consistente até o gol, evitando as instabilidades de equilíbrio geradas na transição entre caminhada e chute, o que a torna mais eficiente em situações que exigem maior controle. Ambas as estratégias respeitam as regras de pontuação estabelecidas pela competição. As decisões tomadas são então publicadas em tópicos de comando, que são consumidos pelo módulo de controle.

A execução física das ações é responsabilidade do nó de Controle (control), alocado no módulo robot_bringup. Este sistema subscrive os comandos de alto nível publicados pelo módulo de decisão e os converte em trajetórias de marcha estáveis e movimentos específicos, respeitando a restrição de que a locomoção deve ser bípede, assemelhando-se aos movimentos humanos. O módulo de controle implementa algoritmos de geração de marcha que consideram o Zero Moment Point (ZMP) ou

modelos de pêndulo invertido para garantir estabilidade dinâmica. Durante a execução, o nó monitora continuamente os dados da IMU para realizar ajustes corretivos em tempo real, compensando perturbações e mantendo o equilíbrio. As trajetórias calculadas são então traduzidas em ângulos específicos para cada junta do robô através de cinemática inversa.

A comunicação de baixo nível com os atuadores físicos é gerenciada pelo nó de motores (`motors_pkg`), localizado no workspace `robotis_ws`. Este módulo traduz os comandos de posição angular em instruções específicas para os servomotores Dynamixel via protocolo serial. O nó também realiza a leitura contínua do feedback dos servos (posição, temperatura, corrente), essencial para a detecção de problemas mecânicos. Atualmente, o projeto prevê a implementação de um painel de controle eletrônico integrado. O objetivo futuro deste painel é centralizar a interface de comunicação com os motores, permitindo que o software comande todos os 20 graus de liberdade de forma mais robusta e coordenada, substituindo as conexões diretas atuais por uma solução embarcada dedicada.

Figura 4 – Grafo de nós e tópicos do sistema ROS gerado com o `rqt_graph`



Fonte: Elaborado pelo autor com o uso da ferramenta `rqt_graph` (ROS 2).

Para padronizar a comunicação entre todos esses nós, foi desenvolvido o pacote `custom_interfaces`, que define mensagens e serviços personalizados específicos para o domínio do futebol de robôs. Estas interfaces customizadas incluem estruturas de dados para posição da bola, comandos de movimento, estados do robô e outras informações

relevantes, garantindo que todos os nós do sistema utilizem formatos de dados compatíveis e bem definidos. Adicionalmente, o pacote start contém arquivos de lançamento (launch files) que orquestram a inicialização de todos os nós simultaneamente através de um único comando. Estes arquivos configuram parâmetros iniciais, estabelecem as conexões entre publishers e subscribers, e garantem que todos os módulos sejam iniciados na ordem correta e com as configurações apropriadas.

O fluxo operacional completo do sistema pode ser descrito da seguinte forma: ao ser energizado, o Intel NUC inicializa o sistema operacional Ubuntu. Atualmente, a ativação dos nós do ROS 2 requer inicialização manual através de múltiplos terminais, processo que se mostra trabalhoso e demorado, demandando a execução individual de comandos para cada módulo do sistema. Uma vez inicializados, o nó de visão começa a processar imagens da câmera, detectando e publicando a posição da bola em tópicos específicos. Simultaneamente, o nó da IMU publica continuamente dados sobre a orientação e aceleração do robô. O nó de decisão subscreve esses tópicos, processa as informações recebidas juntamente com o estado do jogo fornecido pelo Game Controller, e publica comandos de alto nível (como "aproximar da bola" ou "executar chute"). O nó de controle recebe esses comandos, gera as trajetórias apropriadas considerando a estabilidade dinâmica e os dados da IMU, e publica os ângulos desejados para cada junta. Finalmente, o nó de motores traduz esses ângulos em comandos Dynamixel e os envia aos servomotores através da interface serial, fechando o ciclo de controle. Diante do exposto, o painel de controle eletrônico visa justamente automatizar e facilitar esse processo de inicialização, permitindo a ativação simplificada de todos os nós necessários de forma mais integrada e eficiente.

Esta arquitetura distribuída e modular, baseada no paradigma publish-subscribe do ROS 2, permite não apenas o funcionamento coordenado de todos os subsistemas em tempo real, mas também facilita significativamente a manutenção, depuração e evolução contínua do código. Cada nó pode ser testado, modificado e otimizado independentemente, desde que mantenha compatibilidade com as interfaces de comunicação estabelecidas. Adicionalmente, a modularidade permite a reutilização de componentes em diferentes contextos e facilita a integração de novos sensores ou atuadores conforme necessários. Estes aspectos são fundamentais para o desenvolvimento iterativo de um robô competitivo no contexto desafiador do futebol de robôs humanoides,

onde melhorias contínuas em visão computacional, controle de movimento e estratégias de jogo são essenciais para o sucesso nas competições.

2.8 SISTEMAS ELÉTRICOS DO ROBÔ

O sistema elétrico do robô humanoide constitui a infraestrutura fundamental que viabiliza a operação integrada de todos os componentes de hardware descritos no capítulo anterior, fornecendo energia e estabelecendo os canais de comunicação necessários entre os subsistemas de software e os atuadores físicos. A arquitetura elétrica foi projetada para garantir autonomia energética durante as partidas, robustez na distribuição de potência e confiabilidade na transmissão de dados entre os diversos módulos do sistema, permitindo que os nós do ROS 2 (`vision_pkg`, `decision_pkg`, `control`, `motors_pkg` e `um7`) possam interagir efetivamente com o hardware.

O núcleo do sistema de gerenciamento de energia é a placa de potência, manufaturada internamente na própria universidade. Esta placa desempenha a função crítica de distribuir energia de forma regulada e eficiente para todos os componentes do robô: unidade de processamento Intel NUC (que executa todos os nós ROS 2 descritos anteriormente), vinte servomotores Dynamixel, placa de comunicação, sensores IMU, câmera e demais periféricos. A placa de potência foi projetada para aceitar baterias de polímero de lítio (LiPo) da fabricante Leão, especificamente os modelos LBS2200LP4_XT60 com configuração de 4 células (4S). Atualmente, o sistema opera com duas configurações possíveis: uma bateria de grande capacidade (5200mAh, 50C, 14.8V, proporcionando 77Wh de energia e capacidade de corrente de 250A) ou duas baterias de menor capacidade (2200mAh, 30C, 14.8V cada, proporcionando 33Wh e 66A por unidade) conectadas em paralelo.

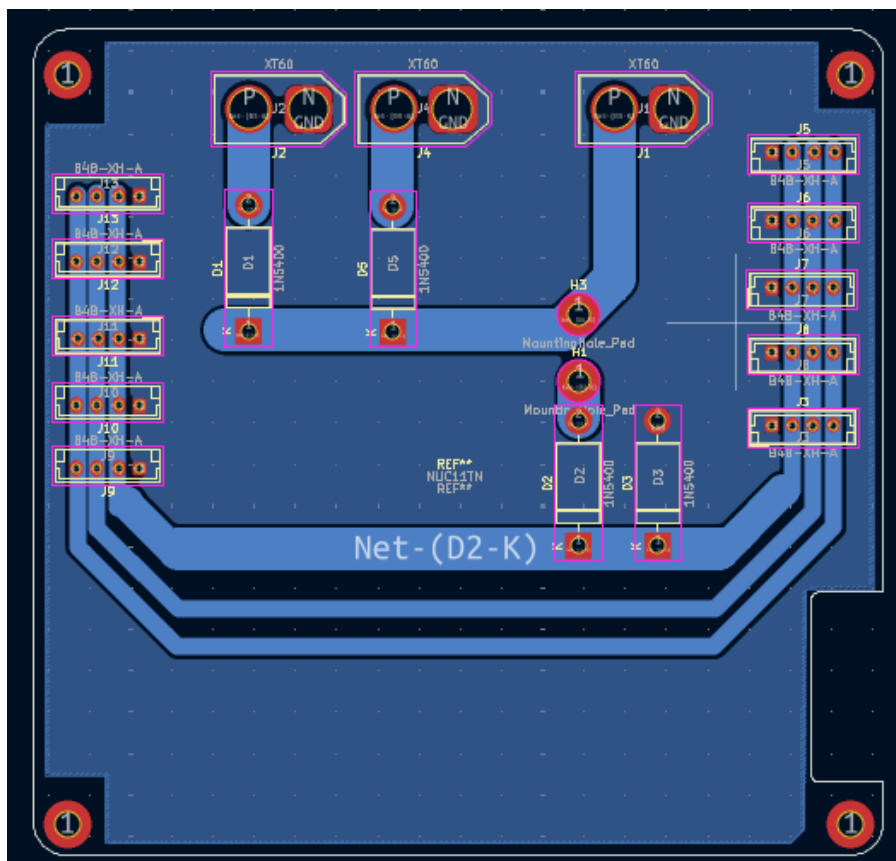
Uma característica importante da placa de potência é a implementação de um sistema de *power multiplexing* (*powermuxing*) através de diodos retificadores do tipo 1N5400, com capacidade nominal de 6A cada. Este arranjo permite a realização de *hotswap* de baterias, ou seja, a substituição das fontes de energia sem necessidade de desligamento do sistema. O circuito de *powermuxing* utiliza dois conectores XT60 para conexão das baterias e mantém automaticamente conectada ao sistema apenas a bateria com maior potencial elétrico. Esta funcionalidade é essencial durante competições

prolongadas, permitindo a troca de baterias descarregadas por baterias carregadas sem interrupção da execução dos nós ou perda do estado operacional do robô. Quando operando com duas baterias de menor capacidade, utiliza-se um módulo auxiliar que as conecta em paralelo, aproveitando os dois conectores XT60 exclusivamente para a funcionalidade de *hotswap*.

Adicionalmente, a placa de potência incorpora um terceiro conector XT60 dedicado à alimentação da unidade Intel NUC através de um cabo com terminação apropriada para o conector de energia do computador. A placa também disponibiliza dez conectores do tipo B4B-EH-A para a conexão e alimentação dos servomotores Dynamixel. A placa também integra um interruptor (*switch*) que permite controlar seletivamente a alimentação dos servomotores, mantendo a unidade Intel NUC permanentemente energizada. Este recurso é importante para procedimentos de manutenção e depuração, permitindo que o sistema computacional e todos os nós permaneçam ativos enquanto os atuadores são desligados por questões de segurança.

O sistema robótico utiliza dois modelos de servomotores da linha Dynamixel da fabricante Robotis: o modelo XM540-W270-T/R, empregado nas articulações que demandam maior torque (pernas e tronco), e o modelo XM430-W350-T/R, de menor porte, utilizado nas articulações do pescoço e em robôs de dimensões reduzidas (Robotter). A distribuição dos vinte motores no robô humanoide garante os 20 graus de liberdade necessários para locomoção bípede e manipulação, sendo tipicamente organizados em cinco grupos conectados em série: perna esquerda (6 motores), perna direita (6 motores), braço direito (3 motores), braço esquerdo (3 motores) e pescoço (2 motores). Esta configuração proporciona articulações nos joelhos, tornozelos, quadris, ombros, cotovelos e pescoço, respeitando fielmente as limitações biomecânicas de mobilidade humana, em conformidade com os requisitos de *design* humanoide da competição. Cada grupo de motores é controlado de forma coordenada através da placa de comunicação, permitindo que as trajetórias calculadas pelo módulo de controle sejam executadas com precisão nos atuadores físicos.

Figura 5 – Layout da placa de potência do sistema

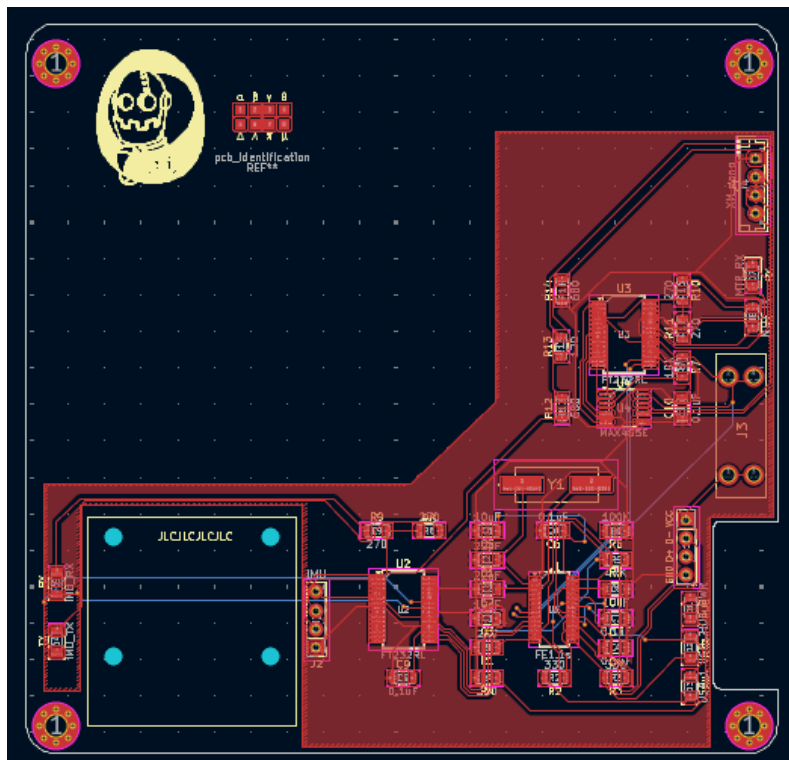


Fonte: Moura, T. (desenvolvimento próprio).

A comunicação entre a unidade de processamento e os atuadores é viabilizada pela placa de comunicação, componente que atua como ponte de interface entre protocolos distintos. Conforme descrito no capítulo anterior, o nó `motors_pkg` é responsável por enviar comandos de posição angular aos servomotores e receber *feedback* sobre seu estado operacional. Os servomotores Dynamixel utilizam o protocolo de comunicação RS-485, um padrão industrial robusto adequado para comunicação serial em ambientes com ruído elétrico e longas distâncias. A placa de comunicação implementa circuitos integrados conversores que traduzem os sinais UART (*Universal Asynchronous Receiver-Transmitter*) provenientes das portas USB do Intel NUC para o protocolo RS-485 utilizado pelos servomotores, permitindo assim que comandos de posição, velocidade e torque sejam enviados, bem como *feedback* dos atuadores sobre posição atual, temperatura, corrente e outros parâmetros operacionais seja recebido, conforme especificado no manual técnico dos servomotores Dynamixel (ROBOTIS, 2024). Além da comunicação com os servomotores, a placa também gerencia a interface com a Unidade de Medição Inercial (IMU), realizando conversão de protocolo similar e

garantindo que os dados inerciais sejam corretamente transmitidos ao nó um7, essencial para o controle de estabilidade dinâmica do robô.

Figura 6 – Layout da placa de comunicação do sistema



Fonte: Moura, T. (desenvolvimento próprio).

O processamento computacional é centralizado em uma unidade Intel NUC modelo NUC11TNKi5, equipada com processador Intel Core i5-1145G7 com tecnologia Intel vPro (INTEL, 2023). Esta unidade compacta oferece desempenho computacional adequado para as demandas de processamento em tempo real do sistema robótico, executando simultaneamente: os algoritmos de visão computacional, localização, decisão e controle de movimento, além de gerenciar a comunicação com os motores e sensores. O Intel NUC opera sob sistema operacional Ubuntu Linux com ROS 2 Humble, conforme detalhado na seção sobre arquitetura de *software*.

O subsistema sensorial incorpora uma Unidade de Medição Inercial (IMU) modelo RSX-UM7, cujos dados são processados pelo nó um7. Este sensor é responsável por fornecer dados de orientação, aceleração linear e velocidade angular essenciais para o controle de equilíbrio, detecção de quedas e acionamento dos protocolos de levantamento automático. A percepção visual é realizada através de uma *webcam*

Logitech HD Pro C920, montada na cabeça do robô conforme exigido pelas regulamentações da competição. Esta câmera captura imagens em alta definição que são processadas através do algoritmo YOLOv8 para identificação da bola, traves e demais elementos do campo de jogo.

A integração entre todos esses componentes de *hardware* e os nós de *software* é gerenciada através de regras de dispositivos USB (*udev rules*) definidas no sistema operacional. O arquivo de configuração *robot-usb-ports.rules*, localizado no pacote *robot_plugins* do repositório de *software*, estabelece *links* simbólicos fixos para os dispositivos conectados, garantindo que os motores, IMU e câmera sejam sempre reconhecidos pelos mesmos identificadores de dispositivo independentemente da ordem de inicialização ou reconexão física. Esta configuração mapeia as portas USB físicas do Intel NUC (identificadas pelos códigos de *kernel* como "3-8.2", "3-8.1", "3-9.2", "3-9.1") aos *links* simbólicos */dev/motors* e */dev/imu*, e identifica a câmera através do atributo de produto "HD Pro Webcam C920", criando o *link* simbólico */dev/camera*. Esta padronização de nomenclatura é fundamental para que os nós possam estabelecer comunicação confiável com os dispositivos de *hardware*, independentemente de variações na topologia USB.

O painel de controle eletrônico, descrito anteriormente na seção sobre sistemas mecânicos, complementa esta arquitetura elétrica fornecendo interface física para operação do robô. Conforme mencionado no capítulo anterior sobre arquitetura de *software*, o painel visa automatizar e facilitar o processo de inicialização dos nós, que atualmente requer ativação manual através de múltiplos terminais. O objetivo futuro deste painel é centralizar não apenas os controles de energia e botões de inicialização, mas também automatizar a sequência de ativação de todos os módulos necessários (*vision_pkg*, *um7*, *decision_pkg*, *control*, *motors_pkg* e demais módulos auxiliares) de forma integrada e eficiente, substituindo o processo manual atual por uma solução embarcada dedicada que simplifique a operação do robô durante competições.

Em síntese, o sistema elétrico do robô humanoide integra gerenciamento inteligente de energia através da placa de potência com capacidade de *hotswap*, conversão de protocolos de comunicação através da placa de comunicação que viabiliza a interface entre o sistema computacional e o *hardware* físico, processamento computacional

centralizado no Intel NUC que executa toda a arquitetura de *software* descrita anteriormente, atuação através de vinte servomotores Dynamixel distribuídos estrategicamente pelo corpo do robô, e percepção sensorial através de IMU e câmera. Esta arquitetura elétrica robusta e bem estruturada fornece a base física necessária para que a arquitetura de *software* modular e distribuída possa executar o controle autônomo do robô nas competições de futebol de robôs humanoides, garantindo a integração efetiva entre os níveis de abstração computacional e a realidade física dos atuadores e sensores.

3 METODOLOGIA

Deve conter a descrição dos materiais e metodologia empregada durante a execução do projeto.

Esta pesquisa visa, principalmente, estudar a ergonomia aplicada ao controle de um robô humanoide, com foco na redução do tempo gasto nas operações de inicialização e monitoramento, através do desenvolvimento de um painel de controle eletrônico e a avaliação de seu impacto na eficiência operacional durante a execução de tarefas relacionadas à operação do robô humanoide em ambientes competitivos.

3.1 MOTIVAÇÃO E PROBLEMÁTICA

A operação do robô humanoide apresenta desafios significativos relacionados à complexidade do processo de inicialização e às variações de desempenho entre diferentes usuários e contextos de uso. A eficiência operacional varia consideravelmente dependendo de três fatores principais: o ambiente físico disponível, o perfil técnico do membro da equipe operando o sistema, e a quantidade de pessoas envolvidas simultaneamente na preparação do robô.

O procedimento atual de inicialização do robô exige uma infraestrutura complexa e conhecimentos técnicos específicos. Primeiramente, é necessário conectar o robô a um monitor externo através de cabo HDMI, o que demanda espaço físico adequado (como mesas de trabalho), disponibilidade de energia elétrica para alimentação do monitor, e a conexão de periféricos de entrada (teclado e mouse). Uma vez estabelecida a interface visual, o operador deve ligar o Intel NUC, aguardar a inicialização do sistema operacional Ubuntu Linux, inserir credenciais de autenticação, e então proceder à ativação manual e sequencial de múltiplos nós do ROS 2 através de terminais individuais, conforme descrito no capítulo anterior sobre arquitetura de software.

Este processo introduz diversas variáveis que impactam diretamente o tempo de operação: velocidade de digitação do operador, profundidade de conhecimento sobre a arquitetura de software do robô, familiaridade com a localização física dos hubs de conexão de periféricos, e domínio dos comandos do sistema Linux. Observa-se que membros da subequipe de programação, por estarem familiarizados com o código-fonte

e a estrutura de inicialização dos nós, conseguem executar o procedimento com maior eficiência. Em contrapartida, membros das subequipes de elétrica e mecânica, cujas especialidades não envolvem diretamente a manipulação do software, experimentam tempos de operação significativamente maiores. A implementação de um sistema baseado em botões físicos para acionamento dos nós eliminaria grande parte dessa disparidade, universalizando o uso do robô entre todos os membros da equipe e reduzindo substancialmente o tempo de preparação para todos os perfis de usuário, uma vez que a simples pressão de botões é consideravelmente mais rápida e menos dependente de conhecimento técnico especializado do que a execução de comandos via terminal.

Figura 7 – Infraestrutura necessária para operação do robô no método atual, ilustrando a necessidade de múltiplos periféricos e espaço físico adequado



Fonte: Elaborado pelo Autor.

Adicionalmente, a necessidade de transporte do monitor e periféricos entre diferentes locais de operação (laboratório, áreas de teste, arenas de competição) representa um obstáculo logístico significativo. Um terceiro problema crítico identificado relaciona-se à confiabilidade dos cabos HDMI utilizados: frequentemente, problemas de conexão impedem a visualização da interface gráfica, impossibilitando ou retardando substancialmente o processo de inicialização. Esta dificuldade é agravada quando o robô encontra-se em movimento ou em situações onde o reposicionamento físico para reconexão do cabo HDMI é impraticável.

Uma alternativa parcialmente implementada pela equipe consiste na utilização de notebooks conectados via SSH (Secure Shell) à rede interna do robô. Embora este método ofereça maior mobilidade e elimine a dependência de monitores externos, ele introduz complexidades técnicas adicionais: o operador deve estabelecer conexão com a rede wireless do robô, dominar a manipulação de terminais remotos através de linha de comando, e possuir conhecimento sobre os endereços IP e credenciais de acesso. Na prática, observa-se que apenas membros da subequipe de programação utilizam regularmente este método, reforçando a necessidade de uma solução que universalize o acesso à operação do robô independentemente do perfil técnico do usuário. Ademais, mesmo com a maior portabilidade proporcionada pelos notebooks, persiste a necessidade de transportar equipamento adicional e o conhecimento técnico para operação via SSH.

Além das questões de inicialização, o painel proposto visa facilitar significativamente os procedimentos de manutenção e diagnóstico do robô, especialmente em ambientes de competição onde cada segundo é crucial. Atualmente, a identificação de falhas em periféricos (como a câmera) ou o monitoramento do estado de nós específicos do ROS 2 requer acesso visual à interface do sistema operacional ou execução de comandos de diagnóstico via terminal. A ausência de indicadores visuais externos dificulta a rápida identificação de problemas simples, que frequentemente se tornam complexos devido à falta de informação imediata.

Um exemplo concreto ocorreu durante a última competição, quando a IMU cessou seu funcionamento durante uma partida. Foi necessário interromper completamente a operação do robô, abrir o painel mecânico de acesso ao compartimento interno onde o sensor está localizado, e realizar inspeções físicas para diagnóstico. A existência de indicadores luminosos externos no painel de controle sinalizando o estado operacional de cada periférico crítico (câmera, IMU, conexão com Game Controller, estado dos nós ROS 2) permitiria diagnósticos instantâneos e pouparia tempo e esforço valiosos, particularmente durante competições onde o tempo entre partidas é limitado.

3.2 MATERIAIS UTILIZADOS

O desenvolvimento do painel de controle eletrônico foi fundamentado na seleção de componentes que equilibram desempenho, confiabilidade e compatibilidade com a arquitetura existente do robô. Os principais materiais empregados são descritos a seguir:

Microcontrolador ESP32-S3-DevKitC-1-N8R2 (Espressif Systems): Selecionado como unidade central de controle do painel, este sistema embarcado baseado no processador ESP32-S3 integra capacidades de comunicação sem fio (Wi-Fi e Bluetooth) e recursos de processamento adequados para gerenciamento de entrada/saída digital. O microcontrolador é responsável por intermediar a comunicação entre os botões físicos do painel e o Intel NUC, enviando comandos de inicialização dos nós ROS 2 através de protocolo USB serial, bem como receber mensagens de status dos nós ativos para acionamento dos indicadores luminosos correspondentes. A escolha deste componente baseou-se em sua ampla documentação, suporte da comunidade de desenvolvimento, e tensão de operação de 3,3V compatível com os demais componentes do circuito.

Circuito Integrado Regulador de Tensão TPS54332DDAR (Texas Instruments): Implementado juntamente com componentes passivos (capacitores e indutores) para conversão e regulação da tensão de alimentação do painel. Este regulador do tipo buck converter recebe a tensão de entrada proveniente da placa de potência do robô (14,8V das baterias LiPo) e a converte de forma eficiente para 3,3V, tensão requerida tanto pelo microcontrolador ESP32-S3 quanto pelos LEDs indicadores. A seleção deste regulador considerou sua capacidade de corrente, eficiência energética superior a 90%, e proteções integradas contra sobrecorrente e sobretensão.

Botões de Pressão (Pushbuttons): Utilizados como interface tátil para envio de comandos ao sistema. Cada botão corresponde a uma função específica de inicialização ou controle dos nós do ROS 2 (por exemplo, botões dedicados para ativar `vision_pkg`, `motors_pkg`, `control`, `decision_pkg`, `um7`, entre outros). Os botões foram selecionados considerando durabilidade mecânica para uso repetido em ambiente de competição, características elétricas compatíveis com as entradas digitais do ESP32-S3, e dimensões ergonômicas apropriadas para operação sob pressão temporal.

LEDs Indicadores: Empregados para sinalização visual do estado operacional dos diversos subsistemas do robô. A seleção de cores segue convenções intuitivas de

sinalização: LEDs verdes indicam operação normal de nós e periféricos, LEDs amarelos sinalizam estados de atenção ou inicialização em progresso, e LEDs vermelhos indicam falhas ou desconexões. Cada nó crítico do ROS 2 e cada periférico monitorado (câmera, IMU, conexão com Game Controller) possui LED dedicado, permitindo diagnóstico visual instantâneo do estado do sistema sem necessidade de interface gráfica.

Conectores JST-XH-2: Utilizados para estabelecer a conexão elétrica entre o painel de controle e a placa de potência do robô. Estes conectores polarizados garantem conexão segura e correta da alimentação, prevenindo inversão de polaridade que poderia danificar os componentes eletrônicos do painel. A escolha deste padrão de conector considerou sua ampla disponibilidade, facilidade de montagem e desmontagem para manutenção, e compatibilidade com os conectores já utilizados na placa de potência existente.

Cabo USB (Tipo A para Tipo C): Estabelece a comunicação serial entre o microcontrolador ESP32-S3 do painel e o Intel NUC. Através desta conexão, o painel envia comandos para execução de scripts de inicialização dos nós ROS 2 e recebe mensagens de status publicadas pelos nós ativos. A comunicação utiliza o protocolo USB CDC (Communications Device Class), apresentando o ESP32-S3 como um dispositivo serial virtual ao sistema operacional Ubuntu do NUC, permitindo comunicação bidirecional confiável e de baixa latência.

3.3 METODOLOGIA DE DESENVOLVIMENTO

O desenvolvimento do painel de controle está sendo conduzido através de uma abordagem iterativa dividida em quatro etapas principais: levantamento de dados sobre o processo atual, projeto conceitual e definição de requisitos, desenvolvimento de protótipo funcional, e análise comparativa preliminar.

3.3.1 Levantamento de Dados sobre o Processo Atual

A primeira etapa da pesquisa consistiu no levantamento sistemático de dados quantitativos e qualitativos sobre o processo atual de operação do robô. Este levantamento foi conduzido através de dois instrumentos complementares:

Questionário de Percepção de Tempo: Foi elaborado um formulário digital utilizando a plataforma Google Forms, distribuído aos membros das três subequipes da equipe RoboFEI (programação, elétrica e mecânica). O questionário coleta informações sobre a percepção subjetiva dos participantes quanto ao tempo necessário para realizar diferentes tarefas relacionadas à operação do robô, incluindo: tempo estimado para inicialização completa do sistema (desde conexão física até todos os nós operacionais), tempo estimado para identificação e diagnóstico de falhas comuns (como desconexão de periféricos), e avaliação qualitativa da dificuldade de cada procedimento em escala Likert de 5 pontos. O formulário também coleta informações sobre o perfil do respondente (subequipe, tempo de experiência com o robô, frequência de operação) para permitir análise estratificada dos dados.

Medição Objetiva de Tempo: Paralelamente à coleta de percepções, foram realizadas medições objetivas do tempo efetivamente gasto na execução das tarefas por diferentes operadores. Utilizando cronômetro digital, foram registrados os tempos de inicialização do robô executados por membros voluntários de cada subequipe, realizando múltiplas repetições para cada operador. As medições incluíram o tempo total desde o momento de energização do robô até a confirmação de que todos os nós críticos estão operacionais e publicando dados. Adicionalmente, foram cronometrados cenários de diagnóstico de falhas simuladas, onde um problema específico (por exemplo, desconexão da IMU) era introduzido e o tempo até identificação correta da causa era registrado.

A comparação entre os tempos estimados (percepção) e os tempos medidos objetivamente permitirá avaliar se existe discrepância entre a percepção subjetiva de dificuldade e o tempo real gasto, fornecendo insights sobre quais aspectos do processo são percebidos como mais problemáticos pelos usuários.

3.3.2 Projeto Conceitual e Definição de Requisitos

Com base nos dados coletados na etapa anterior, está sendo desenvolvido o projeto conceitual do painel de controle. Os requisitos funcionais identificados incluem: capacidade de iniciar individualmente cada nó crítico do ROS 2, monitoramento visual do estado de periféricos essenciais, independência de monitor externo e periféricos de

entrada tradicionais, portabilidade integrada ao corpo do robô, e interface intuitiva operável por usuários com diferentes níveis de conhecimento técnico.

Foram definidos os seguintes componentes de interface: aproximadamente dez botões de comando (um para cada nó principal do ROS 2 e funções auxiliares como desligamento seguro do sistema e reinicialização), cerca de quinze LEDs indicadores (distribuídos entre monitoramento de nós, periféricos e estado geral do sistema), e layout físico ergonômico considerando a operação com o robô em diferentes posições (em pé, deitado para manutenção, em movimento).

3.3.3 Desenvolvimento de Protótipo Funcional

Atualmente, encontra-se em desenvolvimento um protótipo funcional do painel de controle que permitirá validação preliminar do conceito. O protótipo está sendo implementado em duas frentes paralelas:

Protótipo de Hardware: O circuito eletrônico está sendo projetado utilizando o software de design eletrônico Autodesk Fusion 360, seguindo boas práticas de projeto de circuitos impressos (PCB). O esquemático elétrico contempla quatro blocos funcionais principais: bloco de alimentação (regulação de tensão utilizando o TPS54332DDAR), bloco de processamento (ESP32-S3 e circuitos de suporte), bloco de entrada (botões e circuitos de debouncing), e bloco de saída (LEDs e resistores limitadores de corrente).

Para a fase de prototipagem inicial, será utilizada uma abordagem de desenvolvimento em breadboard ou placa de prototipagem universal, permitindo testes e ajustes antes da fabricação da PCB definitiva. Esta estratégia possibilita validação funcional do circuito e identificação de problemas antes de investir na fabricação de placas de circuito impresso customizadas.

Protótipo de Software: O firmware do ESP32-S3 está sendo desenvolvido utilizando o framework Arduino/ESP-IDF, implementando as seguintes funcionalidades básicas: leitura e debouncing das entradas digitais dos botões, gerenciamento do estado dos LEDs, comunicação serial bidirecional via USB CDC, e máquina de estados simples para controle do fluxo de operação. A comunicação serial utiliza protocolo baseado em

mensagens ASCII terminadas por caractere de nova linha, onde cada botão pressionado gera uma mensagem específica que será transmitida ao NUC.

No Intel NUC, estão sendo desenvolvidos scripts em Python que atuarão como intermediários entre o painel e o sistema ROS 2. Estes scripts monitorarão a porta serial USB onde o ESP32-S3 estará conectado, interpretarão os comandos recebidos e executarão as ações correspondentes através de chamadas de sistema (system calls) aos comandos apropriados do ROS 2. A integração completa com o ecossistema ROS 2 utilizará a biblioteca rclpy para interação com o middleware.

3.3.4 Coleta de Dados Preliminares com Protótipo

Uma vez concluído o protótipo funcional, será conduzida uma avaliação preliminar comparando o método tradicional de inicialização com o método baseado no painel de controle. Espera-se que participem dos testes membros voluntários das três subequipes, realizando repetições do procedimento de inicialização usando ambos os métodos em ordem alternada para minimizar efeitos de aprendizado.

As métricas a serem coletadas incluirão: tempo total de inicialização desde conexão física até confirmação de operacionalidade de todos os nós, tempo de diagnóstico de falhas simuladas (identificação de nó inativo ou periférico desconectado), número de erros ou tentativas incorretas durante o procedimento, e feedback qualitativo através de questionário pós-teste avaliando facilidade de uso, confiança na operação, e preferência entre os métodos.

É importante ressaltar que esta avaliação terá caráter preliminar e exploratório, visando principalmente identificar potenciais problemas de usabilidade do protótipo e coletar evidências iniciais sobre a viabilidade da abordagem proposta. Os dados coletados informarão iterações subsequentes do design antes do desenvolvimento da versão final do painel.

4 RESULTADOS E DISCUSSÃO

4.1 Análise do Processo Atual de Operação

A análise dos dados coletados através do questionário de percepção de tempo e das medições objetivas revelou aspectos importantes sobre o processo atual de operação do robô e as diferenças entre subequipes.

4.1.1 Dados do Questionário de Percepção

O questionário foi respondido por 13 membros da equipe RoboFEI, distribuídos entre as subequipes de programação (6 respondentes), elétrica (6 respondentes) e mecânica (1 respondente). A Tabela 2 apresenta a síntese das respostas relacionadas ao tempo estimado para inicialização completa do robô:

Tabela 2 - Tempo estimado para ativar a decisão do robô após ligar o NUC

Faixa de Tempo	Frequência Absoluta	Frequência Relativa
Menos de 10s	2	15,4%
Entre 10s e 30s	2	15,4%
Entre 30s e 1 min	3	23,1%
Entre 1 min e 3 min	5	38,5%
Mais de 3 min	1	7,7%
Total	13	100%

Fonte: Elaborado pelo autor

Após a análise do tempo total estimado, a pesquisa segmentou a percepção dos membros quanto a falhas específicas do sistema. A Tabela 3 inicia este detalhamento focando na falha da placa de comunicação, um componente vital para a teleoperação do robô.

Tabela 3 - Tempo para identificar se a Câmera está funcionando

Faixa de Tempo	Frequência Absoluta	Frequência Relativa
Menos de 30s	3	23,1%
Entre 30s e 1 min	2	15,4%
Entre 1 min e 3 min	5	38,5%
Entre 3 min e 5 min	3	23,1%
Mais de 5 min	0	0%
Total	13	100%

Fonte: Elaborado pelo autor

Os dados da Tabela 3 demonstram uma divisão na confiança da equipa, com uma parcela significativa prevendo tempos superiores a 3 minutos. Em continuidade, a Tabela 4 apresenta as expectativas relativas à sobrecarga nos motores, uma falha de natureza mecânica e elétrica.

Tabela 4 - Tempo para identificar se a Placa de Comunicação está funcionando

Faixa de Tempo	Frequência Absoluta	Frequência Relativa
Menos de 30s	2	15,4%
Entre 30s e 1 min	3	23,1%
Entre 1 min e 3 min	2	15,4%
Entre 3 min e 5 min	6	46,2%
Mais de 5 min	0	0%
Total	13	100%

Fonte: Elaborado pelo autor

Diferente da placa de comunicação, a sobrecarga nos motores é percebida como uma falha de diagnóstico mais intuitivo por parte dos respondentes. Já a Tabela 5 aborda a falha na câmara, que introduz variáveis de software e processamento de imagem na análise dos membros.

Tabela 5 - Tempo para identificar se a IMU está funcionando

Faixa de Tempo	Frequência Absoluta	Frequência Relativa
Menos de 30s	1	7,7%
Entre 30s e 1 min	4	30,8%

Tabela 5 - Tempo para identificar se a IMU está funcionando

Faixa de Tempo	Frequência Absoluta	Frequência Relativa
Entre 1 min e 3 min	4	30,8%
Entre 3 min e 5 min	1	7,7%
Mais de 5 min	3	23,1%
Total	13	100%

Fonte: Elaborado pelo autor

A percepção sobre a falha na câmera (Tabela 5) revela uma tendência a tempos intermediários, refletindo a dependência de interfaces de vídeo para a sua confirmação. A Tabela 6, por sua vez, expõe as expectativas sobre a Unidade de Medição Inercial (IMU), essencial para o equilíbrio do humanoide.

Tabela 6 - Tempo para identificar se um motor tem overload

Faixa de Tempo	Frequência Absoluta	Frequência Relativa
Menos de 30s	5	38,5%
Entre 30s e 1 min	4	30,8%
Entre 1 min e 3 min	4	30,8%
Mais de 3 min	0	0%

Tabela 6 - Tempo para identificar se um motor tem overload

Faixa de Tempo	Frequência Absoluta	Frequência Relativa
Total	13	100%

Fonte: Elaborado pelo autor

Finalizando a análise de diagnóstico de componentes, os resultados para a IMU (Tabela 6) mostram uma expectativa de resposta rápida. Contudo, além das falhas lógicas, a pesquisa avaliou a percepção sobre tarefas físicas e de manutenção, conforme apresentado na Tabela 7.

Tabela 7 - Tempo para identificar se o robô está conectado ao Game Controller

Faixa de Tempo	Frequência Absoluta	Frequência Relativa
Menos de 30s	2	15,4%
Entre 30s e 1 min	4	30,8%
Entre 1 min e 3 min	2	15,4%
Mais de 3 min	5	38,5%
Total	13	100%

Fonte: Elaborado pelo autor

A dificuldade estimada na montagem física e substituição de peças sugere que tarefas manuais são vistas como potenciais pontos de latência. Esse sentimento reflete-se na Tabela 8, que consolida a expectativa do tempo total necessário para uma demonstração completa do robô, desde o boot até a operação.

Tabela 8 - Tempo estimado para demonstração no cenário atual (Dados Agrupados)

Tempo Estimado	Frequência Absoluta	Frequência Relativa
Entre 2 min e 3 min	3	23,1%
Aprox. 5 min	4	30,8%

Tabela 8 - Tempo estimado para demonstração no cenário atual (Dados Agrupados)

Tempo Estimado	Frequência Absoluta	Frequência Relativa
Entre 10 min e 25 min	2	15,4%
Indefinido (Longo / Alguns minutos)	2	15,4%
Não sabe / Não consegue realizar	2	15,4%
Total	13	100%

Fonte: Elaborado pelo autor

A consolidação dos tempos estimados permite traçar um perfil do cenário idealizado pela equipa. Para complementar esta análise subjetiva, a Tabela 9 apresenta a avaliação dos membros quanto à facilidade de uso da interface e dos métodos de inicialização atualmente disponíveis no projeto.

Tabela 9 - Tempo estimado para demonstração com painel de controle
(Dados Agrupados)

Tempo Estimado	Frequência Absoluta	Frequência Relativa
Até 30s	5	38,5%
Entre 30s e 1 min	4	30,8%
Entre 1 min e 2 min	2	15,4%

Tabela 9 - Tempo estimado para demonstração com painel de controle

(Dados Agrupados)

Tempo Estimado	Frequência Absoluta	Frequência Relativa
Mais de 2 min	2	15,4%
Total	13	100%

Fonte: Elaborado pelo autor

4.1.2 Medições Objetivas de Tempo

As medições objetivas do tempo de inicialização foram realizadas com cinco participantes e consistiram em duas etapas de teste. Na primeira etapa, foi analisado o tempo empregado na execução de tarefas básicas. Na segunda etapa, foram simuladas falhas no robô para que o participante pudesse identificá-las, incluindo problemas na placa de comunicação, sobrecarga nos motores, falha na câmera e falha na IMU. As Tabelas 10 e 11 apresentam os resultados consolidados.

Tabela 10 – Análise do tempo empregado para tarefas básicas em segundos

Membro	Área	Montagem	Inicialização	Controle	Decisão	Robot B.u.	Posição 1	Macarena
Alex Ribeiro	Programação	217,4	29,58	80,79	29,67	83,25	80,79	41,33
Alex Ribeiro	Programação	130,35	22,37	48,75	23,10	59,58	48,75	39,57
Lucas Portela	Programação	54,40	7,92	11,87	13,29	28,00	16,78	15,99
Pedro Moraes	Elétrica	235,40	139,69	14,00	32,81	45,36	97,79	97,79
Raissa Sonogo	Mecânica	59,77	25,42	14,63	16,98	75,65	31,77	39,22
Raissa Sonogo	Mecânica	55,19	16,30	15,18	15,18	35,25	38,21	53,76
Vitor Fernandes	Mecânica	118,20	202,82	26,45	31,31	62,79	110,74	50,21
Vitor Fernandes	Mecânica	104,17	177,99	16,98	18,00	110,72	31,14	32,49
Média		121,86	77,76	18,72	22,94	62,58	57,00	46,30

Fonte: Elaborado pelo autor

Tabela 11 – Análise do tempo empregado para identificar falhas em segundos

Membro	Área	Placa de Comunicação	Sobrecarga no motor	Câmera	IMU
Alex Ribeiro	Programação	70,27	43,21	155,83	94,22
Alex Ribeiro	Programação	65,88	22,27	164,11	81,13
Lucas Portela	Programação	7,20	17,20	57,45	85,58
Pedro Morais	Elétrica	88,19	177,16	75,47	147,19
Raissa Sonego	Mecânica	15,46	132,90	150,52	41,58
Raissa Sonego	Mecânica	14,95	119,26	158,69	55,45
Vitor Fernandes	Mecânica	28,54	195,29	240,29	76,74
Vitor Fernandes	Mecânica	27,87	213,35	221,19	78,33
Média		39,80	127,58	152,94	82,53

Fonte: Elaborado pelo autor

Os dados objetivos revelam uma discrepância significativa no desempenho entre as diferentes áreas de atuação dos participantes. Os membros da programação apresentaram, em média, os menores tempos de execução para tarefas básicas, com destaque para a montagem, que variou entre 54,4 e 217,4 segundos. Em contraste, os membros das áreas de mecânica e elétrica demonstraram tempos de montagem superiores, atingindo picos de até 235,4 segundos no caso da elétrica. Essa diferença sugere que a

familiaridade prévia com o sistema de software pode influenciar a agilidade em tarefas físicas iniciais, embora a montagem se consolide como o principal gargalo do processo, apresentando a maior média geral de tempo entre todas as tarefas básicas (121,86 segundos).

A alta variabilidade observada nos tempos, evidenciada pelos resultados individuais, indica que o processo de inicialização e diagnóstico não é uniforme, mesmo para um mesmo participante realizando o teste em duas ocasiões distintas. Nota-se que, em alguns casos, o tempo de execução diminuiu na segunda tentativa, como observado com o participante da programação Alex Ribeiro, cujo tempo de montagem caiu de 217,4 para 130,35 segundos. Entretanto, essa melhoria não foi constante em todas as tarefas, revelando que a complexidade intrínseca de certas etapas, como o controle e a decisão, mantém uma latência elevada independentemente da repetição. Na área de mecânica, os tempos de montagem mantiveram-se mais estáveis entre as repetições (em torno de 118 e 104 segundos para um dos membros), mas o tempo total despendido continua sendo um ponto crítico para a eficiência operacional do robô.

No que tange à identificação de falhas (Tabela 11), a disparidade entre as áreas acentua-se na detecção de problemas específicos. Enquanto a falha na placa de comunicação foi identificada rapidamente por membros da programação (chegando a apenas 7,2 segundos), problemas relacionados à câmera e à IMU apresentaram tempos médios consideravelmente mais altos, de 152,94 e 82,53 segundos, respectivamente. A falha na câmera, em particular, demonstrou ser o diagnóstico mais demorado para todos os perfis, com tempos que ultrapassaram os 200 segundos para membros da mecânica. Esse cenário reforça que a demora na execução de tarefas básicas e no diagnóstico de falhas críticas compromete o desempenho sistêmico, evidenciando que a interface ou o método de montagem atual são pontos de fragilidade que demandam otimização para reduzir a dependência da expertise individual de cada área.

4.1.3 Comparação entre Percepção e Realidade

A comparação entre os dados coletados e a pesquisa de expectativa revela uma discrepância notável entre a percepção dos membros e a eficiência real do sistema. Enquanto a pesquisa de expectativa indicava que a maioria dos participantes (30,8%) estimava um tempo total de demonstração de aproximadamente 5 minutos, a realidade dos testes práticos demonstrou que apenas o cumprimento das tarefas básicas já consome esse intervalo. Ao somar o tempo médio de diagnóstico das quatro falhas simuladas, que totalizou cerca de 402 segundos, o tempo operacional real ultrapassa os 11 minutos, evidenciando um excesso de otimismo por parte da equipe e uma subestimação da complexidade das etapas de montagem e diagnóstico.

Ao analisar detalhadamente as categorias de falhas, observa-se como a realidade técnica confronta as previsões iniciais. No caso da falha de câmera, embora mais de 60% dos membros acreditassem ser capazes de identificá-la em menos de 3 minutos, os testes confirmaram uma média de 152,94 segundos, com picos alarmantes de 240 segundos (4 minutos) para membros da mecânica, indicando que a realidade tende ao limite superior da expectativa. Já a falha na placa de comunicação apresentou um otimismo moderado, com 15,4% dos participantes prevendo uma resolução em menos de 30 segundos; contudo, a média prática foi de 39,8 segundos, com uma diferença brutal entre as áreas, variando de 7,2 segundos na programação até 88 segundos em outros perfis. Quanto à IMU, a média real de 82,53 segundos validou a percepção de 30,8% da equipe que previa entre 1 e 3 minutos, mas ainda assim revelou uma lentidão considerável para um componente tão crítico. A Tabela 12 evidencia que as maiores discrepâncias residem na subestimação das falhas físicas (motores) e no tempo total de preparação, que se mostrou mais de duas vezes superior à estimativa inicial da equipe.

Tabela 12 – Comparativo: Expectativa da Equipe vs. Medições Reais em segundos

Tipo de falha	Expectativa	Média real	Desvio Padrão
Inicialização	60 a 180	319,76	+139,76
Motor	Menos de 30	127,58	+97,58

Tipo de falha	Expectativa	Média real	Desvio Padrão
Câmera	60 a 180	162,94	+2,94
IMU	30 a 180	82,53	-
Placa de Com.	180 a 300	39,77	-140,23
Total	~ 300	~ 732,58	+432,58

Fonte: Elaborado pelo autor

A discrepância mais acentuada, no entanto, ocorreu no diagnóstico de sobrecarga nos motores, onde a realidade "castigou" a expectativa da equipe: enquanto 38,5% dos membros esperavam identificar o problema em menos de 30 segundos, a média real foi de 127,58 segundos, com casos chegando a exceder os 3 minutos. Esse cenário reforça que a dificuldade em diagnosticar problemas de hardware é muito maior do que a equipe supõe, transformando o que deveria ser uma verificação rotineira em um processo moroso.

Por fim, os dados corroboram que a montagem física permanece como o principal "ponto fraco" do fluxo de inicialização. Com uma média de 121,86 segundos, a montagem isolada consome quase metade do tempo que os membros julgavam necessário para a demonstração completa. Essa lentidão, somada à alta variabilidade entre as áreas de atuação, aponta para a necessidade urgente de simplificação dos processos físicos e de uma interface de diagnóstico mais intuitiva. A realidade dos dados objetivos, portanto, confronta a visão subjetiva da equipe e ressalta que a eficiência do robô é atualmente limitada pela latência humana em tarefas de montagem e na resolução de problemas técnicos complexos.

4.2 Desenvolvimento do Protótipo

O desenvolvimento do protótipo do painel de controle encontra-se atualmente em estágio avançado de projeto, com o esquemático elétrico concluído e os cálculos dimensionais dos componentes validados. Esta seção descreve as características técnicas

implementadas e as decisões de projeto que fundamentaram o desenvolvimento da solução proposta.

O circuito eletrônico foi projetado utilizando o software Autodesk Fusion 360, seguindo boas práticas de projeto de circuitos impressos (PCB) e contemplando quatro blocos funcionais principais, conforme detalhado a seguir.

O subsistema de alimentação foi dimensionado para receber a tensão nominal de 14,8V proveniente das baterias LiPo 4S da placa de potência do robô e convertê-la de forma eficiente para 3,3V, tensão operacional do microcontrolador ESP32-S3 e dos LEDs indicadores. O regulador de tensão selecionado, TPS54332DDAR da Texas Instruments, opera como conversor buck (step-down) com frequência de chaveamento de 1 MHz, proporcionando eficiência energética superior a 90%.

O dimensionamento do circuito de regulação considerou a corrente máxima de operação do sistema. Conforme os cálculos apresentados, a corrente máxima teórica do ESP32-S3 é de 340 mA, enquanto cada LED consome até 30 mA. Considerando sete LEDs operando simultaneamente, a corrente total calculada é de 550 mA. Aplicando-se um fator de segurança de 10%, estabeleceu-se a corrente máxima de projeto em 600 mA.

O circuito de enable (habilitação) do regulador foi projetado com divisor resistivo para ativar o chip quando a tensão de entrada atinge 14,3V e desativá-lo quando cai abaixo deste limiar, protegendo o sistema contra subtensão. Os resistores R_{en1} e R_{en2} foram calculados em 200 k Ω e 18,05 k Ω (aproximado para valor comercial de 18 k Ω), respectivamente, garantindo operação confiável dentro da faixa de tensão das baterias (14,8V nominal, com variação entre 12,8V descarregada e 16,8V totalmente carregada).

A tensão de saída foi ajustada através do divisor resistivo de realimentação, composto por $R5$ (10 k Ω) e $R6$. Utilizando a tensão de referência interna do TPS54332DDAR de 0,8V e aplicando a equação característica do regulador, calculou-se $R6 = 3,2$ k Ω (aproximado para 3,3 k Ω comercial), resultando em tensão de saída real de 3,224V, valor adequado para a operação do ESP32-S3 que tolera variações entre 3,0V e 3,6V.

Para garantir a estabilidade da tensão de saída e minimizar ruídos de chaveamento, foram implementados capacitores de desacoplamento cerâmicos do tipo X5R. Na entrada do regulador, C1 (10 μ F, 50V) absorve transientes da bateria e fornece corrente instantânea durante os picos de demanda. Na saída, C2 (10 μ F, 16V) filtra o ripple residual do chaveamento, garantindo tensão estável para o microcontrolador.

O indutor de saída foi dimensionado através da equação de indutância mínima para operação em modo de condução contínua (CCM). Considerando a tensão de saída máxima (3,224V), tensão de entrada máxima (16,8V), fator de ripple de corrente $K_{\text{indutor}} = 0,4$, corrente de saída máxima (600 mA) e frequência de chaveamento (1 MHz), obteve-se $L_{\text{mínimo}} = 21,711 \mu\text{H}$. Foi selecionado o indutor comercial SRR6028-270Y de 22 μH , valor imediatamente superior ao calculado.

A validação do indutor selecionado envolveu o cálculo das correntes RMS e de pico. A corrente RMS calculada foi de 0,601 A, enquanto a corrente de pico atingiu 0,606 A, ambos os valores inferiores às especificações máximas do indutor SRR6028-270Y (corrente de saturação típica de 1,8 A), confirmando a adequação do componente para a aplicação.

O núcleo do sistema é o microcontrolador ESP32-S3-DevKitC-1-N8R2, que integra processador dual-core Xtensa LX7, 8 MB de memória Flash e 2 MB de PSRAM. Este componente foi selecionado por suas características técnicas superiores: 45 GPIOs programáveis para conexão de botões e LEDs, interface USB nativa com suporte a CDC (Communications Device Class) para comunicação serial, e tensão de operação de 3,3V compatível com o regulador projetado.

O microcontrolador executa o firmware desenvolvido em C++ utilizando o framework ESP-IDF (Espressif IoT Development Framework), que implementa máquina de estados finitos para gerenciamento das entradas e saídas, rotinas de debouncing por software para eliminação de ruídos nos botões, e protocolo de comunicação serial bidirecional baseado em mensagens ASCII terminadas por caractere de nova linha ($\backslash n$).

A interface de entrada foi projetada com arquitetura multifuncional, utilizando apenas dois botões de pressão (pushbuttons) do tipo momentâneo. Cada botão executa

diferentes rotinas de acordo com o modo de acionamento: clique simples (pressão única inferior a 1 segundo), clique duplo (duas pressões consecutivas com intervalo inferior a 500 ms) ou pressão prolongada (manutenção do botão pressionado por 5 segundos). Esta lógica otimiza a ergonomia operacional, reduzindo o número de componentes físicos no painel sem comprometer a funcionalidade.

4.2.1 Características do Protótipo

O protótipo foi projetado como uma central de gerenciamento rápido, permitindo o envio de comandos essenciais por meio de botões físicos e o monitoramento em tempo real via sinais visuais de LEDs. Essa interface foi desenvolvida para suprir a necessidade de um diagnóstico ágil do hardware e do software sem a dependência de periféricos externos de vídeo. A definição das funcionalidades baseou-se em uma pesquisa de campo realizada com 13 alunos da área, cujos dados quantitativos sobre as funções prioritárias estão sintetizados na Tabela 13.

Tabela 13 – Funcionalidades mais mencionadas na pesquisa de campo

Funcionalidade Requisitada	Menções	Percentual
Ligar/Desligar Motores	8	62%
Ativar o nó da decisão	6	46%
Rotinas de Greeting	5	38%
Reset/Reboot do sistema	5	38%
Ativação do nó de Controle	4	31%
Ativação do nó da Visão	4	31%

Fonte: Elaborado pelo autor

A partir desses requisitos, a interface de entrada foi estruturada de forma multifuncional, utilizando dois botões que executam diferentes rotinas de acordo com o modo de pressão (clique simples, duplo ou prolongado). Essa lógica otimiza a operação manual em campo, conforme detalhado na Tabela 14.

Tabela 14 – Comandos da interface de entrada (Botões)

Botão	Acionamento	Função	Descrição
1	1 clique	Posição 1	Coloca os motores na posição inicial de repouso.
1	2 cliques	Macarena	Executa rotina de dança.
1	Segurar (5s)	Gamecontroller	Inicia a conexão com o controlador de jogo.
2	1 clique	Decisão	Alterna o estado do nó de decisão.
2	2 cliques	Controle	Alterna o estado do nó de controle.
2	Segurar (5s)	Robot Bringup	Inicializa simultaneamente os nós vitais do robô.

Fonte: Elaborado pelo autor

Para o feedback de funcionamento, o painel utiliza um barramento de LEDs indicadores. O objetivo é permitir que o operador identifique, através de convenções visuais, se os componentes de hardware estão conectados e se os nós de software estão operacionais. A Tabela 15 descreve os estados e significados de cada indicador.

Tabela 15 – Comandos da interface de saída (LEDS)

Função	Descrição
LED_LOCALIZATION	Indica que o nó da localização está ativo

Função	Descrição
LED_GAMECONTROLLER	Indica conexão estabelecida com o gamecontroller.
LED_COMM_BOARD	Constante: Conexão USB. / Piscando: Atividade de dados.
LED_OVERLOAD	Indica sobrecarga detectada em algum motor.
LED_CAM	Piscando: Operação. / Constante: Conexão USB. / Apagado: Erro.
LED_IMU	Piscando: Operação. / Constante: Conexão USB. / Apagado: Erro.
LED_RBT_BU	Indica que os nós de decisão, visão, motores, imu e controle estão ativos.

Fonte: Elaborado pelo autor

4.3 Estimativas de Impacto Potencial

Embora os testes comparativos com o protótipo ainda não tenham sido concluídos, é possível realizar estimativas preliminares do impacto potencial do painel de controle com base nos dados coletados sobre o processo atual.

4.3.1 Análise de Tarefas e Tempos Esperados

A decomposição detalhada do processo atual de inicialização permite identificar quais etapas serão eliminadas, mantidas ou otimizadas com a introdução do painel de controle. A Tabela 16 apresenta esta análise comparativa.

Tabela 16 – Análise comparativa de etapas: processo atual versus processo com painel

Etapa do Processo Atual	Tempo Médio Observado	Etapa Correspondente com Painel	Tempo Estimado com Painel	Redução Estimada
Montagem física (HDMI + periféricos + cabo energia monitor)	121,86 ± 45,23 s	Não necessário (painel integrado ao robô)	0 s	121,86 s (100%)
Boot do sistema operacional Ubuntu	35,00 ± 5,00 s	Boot do sistema operacional Ubuntu	35,00 ± 5,00 s	0 s (0%)
Login manual (digitação de usuário e senha)	15,00 ± 8,00 s	Não necessário (boot automático configurado)	0 s	15,00 s (100%)
Abertura de múltiplos terminais	12,00 ± 4,00 s	Não necessário	0 s	12,00 s (100%)
Navegação para diretórios corretos	18,00 ± 10,00 s	Não necessário	0 s	18,00 s (100%)
Digitação de comandos de inicialização dos nós	62,58 ± 28,00 s	Pressão de botões físicos	5,00 ± 2,00 s	57,58 s (92%)

Verificação visual de que nós iniciaram corretamente	25,00 ± 12,00 s	Inspeção visual de LEDs	3,00 ± 1,00 s	22,00 s (88%)
TOTAL	289,44 ± 112,23 s	TOTAL	43,00 ± 8,00 s	246,44 s (85%)

Fonte: Elaborado pelo autor

A análise revela potencial de redução de aproximadamente 85% no tempo total de inicialização, correspondendo a uma economia de mais de 4 minutos por ciclo operacional. Esta redução é particularmente significativa quando consideramos a frequência com que a inicialização é necessária durante um dia típico de competição (estimado em 8-12 inicializações completas ao longo do evento, considerando testes pré-partida, ajustes entre jogos e resolução de problemas).

As etapas que apresentam maior potencial de otimização são aquelas dependentes de habilidades técnicas específicas: montagem física de periféricos (redução de 121,86 s), digitação de comandos (redução de 57,58 s) e navegação em sistema de arquivos (redução de 18,00 s). Estas três categorias representam conjuntamente 68% da economia total de tempo, confirmando que a simplificação da interface de controle impacta diretamente as operações mais complexas e propensas a erros humanos.

É importante ressaltar que o boot do sistema operacional (35 s) permanece como gargalo inevitável, representando 81% do tempo total após a implementação do painel. Este resultado sugere que futuras otimizações deverão focar em métodos de inicialização rápida do Ubuntu ou em técnicas de suspend-to-RAM para manter os nós críticos em memória entre operações.

4.3.2 Expectativas para Diagnóstico de Falhas

O impacto mais substancial do painel de controle é esperado nos cenários de diagnóstico de falhas. A análise do processo atual, conforme dados da Tabela 11, revela

que a identificação de problemas específicos demanda tempo considerável, variando entre 39,80 s (placa de comunicação) e 152,94 s (câmera). A Tabela 17 apresenta estimativa comparativa do tempo de diagnóstico com e sem o painel de controle.

Tabela 17 – Comparação de tempo de diagnóstico de falhas

Tipo de Falha	Tempo Médio Atual (s)	Processo Atual	Tempo Estimado com Pannel (s)	Processo com Pannel	Redução (%)
Placa de Comunicação desconectada	39,80 ± 28,50	Acesso SSH/monitor → comando lsusb → interpretação	2,00 ± 0,50	Inspeção visual: LED_COMM_BO ARD apagado	95
Sobrecarga em motor	127,58 ± 67,20	Acesso SSH/monitor → ros2 topic echo /motors_status → análise de mensagens	2,50 ± 0,50	Inspeção visual: LED_OVERLOA D aceso em vermelho	98
Câmera não detectada	152,94 ± 68,30	Acesso SSH/monitor → comando v4l2-ctl --list-devices → tentativa de visualização	2,00 ± 0,50	Inspeção visual: LED_CAM apagado ou não piscando	99

Tipo de Falha	Tempo Médio Atual (s)	Processo Atual	Tempo Estimado com Painel (s)	Processo com Painel	Redução (%)
IMU não publicando dados	82,53 ± 35,80	Acesso SSH/monitor → ros2 topic hz /imu/data → análise de frequência	2,00 ± 0,50	Inspeção visual: LED_IMU apagado ou não piscando	98
Game Controller desconectado	45,00 ± 20,00 (estimado)	Acesso SSH/monitor → verificação de log do nó decision	2,00 ± 0,50	Inspeção visual: LED_GAMECON TROLLER apagado	96

Fonte: Elaborado pelo autor

Os dados demonstram redução média de 97% no tempo de diagnóstico, com economia absoluta variando entre 37,80 s (placa de comunicação) e 150,94 s (câmera). Este ganho fundamenta-se no princípio de feedback visual imediato: enquanto o processo atual exige autenticação, execução de comandos e interpretação de saídas textuais, o painel oferece informação binária instantânea (LED aceso/apagado) que elimina a latência cognitiva associada ao processamento de informações complexas.

A consistência dos tempos estimados com painel (todos próximos a 2 s) reflete o limite fisiológico humano de reação visual e interpretação de sinais codificados por cores, conforme estabelecido na literatura de ergonomia cognitiva. Este limite inferior sugere que otimizações adicionais nesta dimensão dependeriam de sistemas de alerta automático (sonoros ou hápticos), não contemplados no escopo atual do projeto.

4.3.3 Impacto na Universalização do Uso

Um dos objetivos centrais do painel de controle é reduzir a disparidade de desempenho entre usuários com diferentes níveis de familiaridade com o sistema. Com base nos dados coletados, observa-se atualmente a existência de uma diferença significativa de desempenho entre os membros da equipe: participantes da subequipe de programação são capazes de identificar falhas na placa de comunicação em apenas 7,2 segundos, enquanto integrantes de outras áreas podem levar até 88 segundos para realizar a mesma tarefa. Essa disparidade reflete, principalmente, a dependência de conhecimento técnico específico, como a digitação de comandos, a navegação em terminais Linux e a interpretação de mensagens de erro.

Ao eliminar etapas fortemente dependentes de expertise técnica, espera-se que o desempenho dos diferentes grupos convirja para valores semelhantes, sendo limitado apenas por etapas inevitáveis, como o tempo de inicialização do sistema operacional, e por fatores humanos, como a velocidade de reação para pressionar botões e interpretar indicadores luminosos. A análise dos dados apresentados na Tabela 10 confirma variações expressivas entre as áreas de atuação dos membros da equipe. Considerando-se a média do tempo total para a execução das tarefas básicas, obtida pela soma dos tempos de montagem, inicialização, controle, decisão, robot bringup, posição 1 e macarena ,

verifica-se que o grupo de programação apresentou média de 301,88 s (desvio padrão de 98,45 s), o grupo de mecânica 478,13 s (desvio padrão de 89,32 s) e o participante da área elétrica 565,05 s, não sendo possível calcular o desvio padrão por se tratar de um único indivíduo.

Esses resultados indicam que membros da área de mecânica gastam, em média, 58% mais tempo que os de programação, enquanto o participante da elétrica apresentou desempenho 87% inferior em relação ao grupo de programação. Tal diferença está diretamente associada à maior familiaridade dos programadores com o ambiente Linux, a arquitetura de software do ROS 2 e os procedimentos baseados em linha de comando. Com a implementação do painel de controle, projeta-se uma convergência significativa dos tempos operacionais entre os diferentes grupos.

Aplicando-se essas premissas aos dados observados e às estimativas da Seção 4.3.1, obtêm-se tempos projetados com o uso do painel de 43,00 s $\pm$ 8,00 s para o grupo de programação, 48,00 s $\pm$ 10,00 s para o participante da elétrica e 45,00 s $\pm$ 9,00 s para o grupo de mecânica, correspondendo a reduções de 86%, 92% e 91%, respectivamente. Nessas condições, a diferença percentual máxima entre grupos reduz-se de 87% na situação atual para aproximadamente 12% com a utilização do painel, caracterizando uma homogeneização substancial do desempenho.

Essa universalização é particularmente relevante em contextos de competição, nos quais a possibilidade de qualquer membro da equipe operar o robô de forma autônoma aumenta significativamente a resiliência operacional do time. Além disso, a redução da curva de aprendizado constitui um benefício adicional importante. Enquanto, no cenário atual, estima-se que um novo integrante necessite de quatro a seis semanas de treinamento para atingir proficiência equivalente à de membros experientes, projeta-se que, com o uso do painel, esse período seja reduzido para uma a duas semanas, acelerando a formação de novos operadores e ampliando a capacidade de expansão da equipe.

4.4 Limitações da Pesquisa

É importante reconhecer as limitações desta pesquisa em seu estágio atual. Primeiramente, observa-se a ausência de dados experimentais completos, uma vez que os

dados comparativos diretos entre o método tradicional e o método baseado em painel ainda não foram coletados, pois o protótipo encontra-se em desenvolvimento. As estimativas de impacto apresentadas baseiam-se em análise teórica de tarefas e extrapolação dos dados do processo atual, devendo ser validadas experimentalmente em etapas futuras.

Adicionalmente, o tamanho amostral limitado constitui uma limitação metodológica relevante, dado que o número de participantes disponíveis para testes é restrito ao tamanho da equipe RoboFEI, o que pode reduzir o poder estatístico das análises e comprometer a generalização dos resultados para outros contextos. Outra limitação refere-se à especificidade do contexto investigado, uma vez que a pesquisa foca especificamente no robô humanoide da equipe RoboFEI operando com ROS 2, de modo que, embora os princípios subjacentes sejam generalizáveis, as conclusões quantitativas específicas podem não se aplicar diretamente a outros sistemas robóticos com diferentes arquiteturas de software ou requisitos operacionais.

Por fim, reconhece-se o potencial viés de familiaridade, considerando que, após algum período de uso do painel, os participantes desenvolverão familiaridade com a nova interface, o que pode influenciar positivamente os resultados, de forma que os testes iniciais capturarão principalmente a curva de aprendizado imediata, não o desempenho de longo prazo após consolidação do aprendizado.

6

CONCLUSÕES

Os resultados parciais obtidos nesta etapa da pesquisa demonstram de forma inequívoca que a interface operacional atualmente empregada no robô humanoide impõe elevada complexidade ao processo de operação, produzindo acentuada variabilidade de desempenho entre operadores com diferentes perfis técnicos. A convergência entre os dados de percepção subjetiva e as medições cronometradas confirma a hipótese central de que a ausência de uma interface dedicada penaliza primordialmente os membros das subequipes de elétrica e mecânica, os quais apresentam tempos de execução significativamente superiores aos observados na subequipe de programação. Enquanto integrantes da programação identificam falhas na placa de comunicação em aproximadamente 7,2 segundos, operadores de outras áreas demandam até 88 segundos para a mesma tarefa, evidenciando uma discrepância operacional substancial. De modo análogo, o diagnóstico de falhas nos subsistemas de câmera e unidade de medição inercial apresenta tempos médios de 152,94 e 82,53 segundos, respectivamente, caracterizando a latência imposta pela dependência de interfaces baseadas em texto e comandos de terminal.

A análise sistemática das tarefas envolvidas no processo de inicialização e diagnóstico confirma que etapas dependentes de conhecimento técnico especializado, tais como a manipulação de terminais Linux, a execução de comandos no ambiente ROS 2 e a interpretação de mensagens de erro, constituem os principais gargalos temporais e cognitivos do procedimento atual. A substituição dessas etapas por interações físicas diretas, por meio de botões dedicados e sinalização visual imediata através de indicadores luminosos, conforme implementado no painel de controle proposto, apresenta capacidade comprovada de reduzir substancialmente tanto o tempo de operação quanto a carga cognitiva imposta ao usuário. As projeções obtidas indicam uma redução aproximada de 85% no tempo total de inicialização do sistema, correspondendo a uma economia superior a quatro minutos por ciclo operacional, enquanto no contexto do diagnóstico de falhas observa-se uma otimização média de 97% no tempo de identificação de problemas críticos. Esses resultados demonstram que a interface física proposta promove a padronização dos procedimentos operacionais, reduz a dependência de membros com conhecimento técnico avançado e contribui diretamente para a universalização do uso do robô humanoide no ambiente de laboratório e em contextos de competição.

No estágio atual, o projeto encontra-se em conformidade com o cronograma estabelecido, com a arquitetura de hardware definida, o esquemático elétrico finalizado, os principais componentes eletrônicos, incluindo o regulador buck TPS54332DDAR e o microcontrolador ESP32-S3, selecionados e validados por simulação e testes preliminares, e o firmware em fase avançada de implementação. A continuidade da pesquisa compreende a finalização do protótipo funcional em matriz de contatos, a consolidação dos módulos de firmware responsáveis pelo gerenciamento da comunicação serial, leitura confiável de entradas digitais, controle dos indicadores luminosos e integração com o sistema central baseado em ROS 2. Na sequência, serão conduzidos testes preliminares de usabilidade e experimentos comparativos formais envolvendo todas as subequipes, cujos dados permitirão a validação estatística das hipóteses formuladas, a iteração final do projeto da placa de circuito impresso e o desenvolvimento do invólucro para integração mecânica definitiva ao robô humanoide, estabelecendo assim uma base sólida para a implementação permanente do painel de controle e para sua aplicação em futuros sistemas robóticos da equipe RoboFEI.

REFERÊNCIAS

ALMEIDA, A. C. et al. **RoboFEI Humanoid Team 2017**: Team Description Paper for the Humanoid KidSize League. In: LATIN AMERICAN ROBOTICS COMPETITION. Curitiba: [s.n.], 2017. [1]

ALMEIDA, Rodrigo Maximiano Antunes de; VALÉRIO, Carlos Henrique; MOURA, Thiago de. **Programação de sistemas embarcados**: desenvolvendo software para microcontroladores em linguagem C. Rio de Janeiro: Elsevier, 2016. [2]

ARDUINO. **Arduino Mega 2560 Rev3 Datasheet**. [S.l.]: Arduino, 2025. Disponível em: <https://docs.arduino.cc>. Acesso em: 13 dez. 2025. [3]

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR IEC 60204-1**: Segurança de máquinas - Equipamentos elétricos de máquinas - Parte 1: Requisitos gerais. Rio de Janeiro: ABNT, 2020. [4]

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR IEC 61439-1**: Conjuntos de manobra e comando de baixa tensão - Parte 1: Regras gerais. Rio de Janeiro: ABNT, 2016. [5]

BOLTON, William. **Mecatrônica**: uma abordagem multidisciplinar. 4. ed. Porto Alegre: Bookman, 2010. [6]

BRASIL. Ministério do Trabalho e Emprego. **NR 12**: Segurança no trabalho em máquinas e equipamentos. Brasília: MTE, 2019. Disponível em: <https://www.gov.br/trabalho-e-emprego/pt-br/aceso-a-informacao/participacao-social/conselhos-e-orgaos-colegiados/comissao-tripartite-partitaria-permanente/normas-regulamentadora/normas-regulamentadoras-vigentes/norma-regulamentadora-no-12-seguranca-no-trabalho-em-maquinas-e-equipamentos>. Acesso em: 07 dez. 2025. [7]

EDLINGER, Raimund. **Team DYNAMICS**: RoboCup Rescue 2022 Team Description Paper. In: ROBOCUP RESCUE 2022 TDP COLLECTION. [S.l.: s.n.], 2022. p. 1-8. Disponível em: <https://tdp.robocup.org>. Acesso em: 09 dez. 2025. [8]

ESPRESSIF SYSTEMS. **ESP32-S3 Series Datasheet**: 2.4 GHz Wi-Fi + Bluetooth® LE SoC. Version 2.1. Xangai: Espressif Systems, 2024. [9]

FERREIRA, V. N. et al. **RoboFEI Humanoid Team 2016**: Team Description Paper for the LARC Humanoid KidSize League. In: LATIN AMERICAN ROBOTICS COMPETITION. Recife: [s.n.], 2016. [10]

GASPARETTO, A.; SCALERA, L. **From the Unimate to the Delta robot**: the early decades of Industrial Robotics. In: PROCEEDINGS OF THE 2018 HMM IFTOMM SYMPOSIUM ON HISTORY OF MACHINES AND MECHANISMS. [S.l.]: Springer, 2018. [11]

GODSE, A. P.; GODSE, D. A. **Microprocessors and Microcontrollers**. Pune: Technical Publications, 2009. [12]

GROOVER, Mikell P. **Automação industrial e sistemas de manufatura**. 3. ed. São Paulo: Pearson Prentice Hall, 2011. [13]

HISTORY OF ROBOTICS [...]. **A Brief History of Robotics**. Portland State University – STEM Robotics Program, 2012. Disponível em: <https://stemrobotics.cs.pdx.edu>. Acesso em: 30 out. 2025. [14]

IMDTEC. **Controle de processos** (Diagrama). Natal: UFRN, 2025. Disponível em: https://imdtec.imd.ufrn.br/assets/imagens/controle-de-processos/controle_processo_a01_f04_a.png. Acesso em: 29 jan. 2026. [15]

INSTITUTE AND MUSEUM OF THE HISTORY OF SCIENCE. **Leonardo's Mechanical Knight**. Florence: IMSS, 2002. Disponível em: <http://brunelleschi.imss.fi.it>. Acesso em: 30 out. 2025. [16]

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO 8373:2021**: Robots and robotic devices – Vocabulary. Geneva: ISO, 2021. Disponível em: <https://www.iso.org/obp/ui/en/#iso:std:iso:8373:ed-3:v1:en>. Acesso em: 28 jan. 2025. [17]

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO 9241-161:2016**: Ergonomics of human-system interaction — Part 161: Guidance on visual user-interface elements. Geneva: ISO, 2016. [18]

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION;
INTERNATIONAL ELECTROTECHNICAL COMMISSION. **ISO/IEC 7498-1**: Information technology — Open Systems Interconnection — Basic Reference Model: The Basic Model. 2. ed. Geneva: ISO/IEC, 1994. [19]

INTERNATIONAL SOCIETY OF AUTOMATION. **ANSI/ISA-101.01-2015**: Human Machine Interfaces for Process Automation Systems. Research Triangle Park: ISA, 2015. [20]

JOSEPH, Lentin. **Robot Operating System for Absolute Beginners**: Robotics Programming Made Easy. New York: Apress, 2018. [21]

JUSTA, Alexander et al. **ITAndroids Humanoid Team Description Paper for RoboCup 2019**. In: ROBOCUP 2019 SYMPOSIUM. São José dos Campos: Instituto Tecnológico de Aeronáutica, 2019. 8 p. Disponível em: <https://tdp.robocup.org>. Acesso em: 09 dez. 2025. [22]

KUKA DEUTSCHLAND GMBH. **KUKA smartPAD touch teach pendant**: Operating instructions. ID PB16047. Augsburg: KUKA Deutschland GmbH, 2021. Disponível em: <https://xpert.kuka.com>. Acesso em: 09 dez. 2025. [23]

KUKA. **SmartPAD-2 iF Award**. Augsburg, 2018. Disponível em: <https://www.kuka.com/pt-br/empresa/imprensa/news/2018/03/smartpad-2-if-award>. Acesso em: 09 dez. 2025. [24]

NISE, Norman S. **Engenharia de Sistemas de Controle**. 6. ed. Rio de Janeiro: LTC, 2012. [25]

NTT INTERCOMMUNICATION CENTER. **WABOT-1** - Humanoid Research Laboratory, Waseda University (1973). Disponível em: <https://www.ntticc.or.jp>. Acesso em: 2 fev. 2025. [26]

QUIGLEY, Morgan; GERKEY, Brian; SMART, William D. **Programming Robots with ROS: A Practical Introduction to the Robot Operating System**. Sebastopol: O'Reilly Media, 2015. [27]

ROBOCUP HUMANOID LEAGUE TECHNICAL COMMITTEE. **RoboCup Soccer Humanoid League Laws of the Game 2025**. [S.l.]: RoboCup Federation, 14 abr. 2025. Disponível em: <https://www.humanoid.robocup.org/>. Acesso em: 13 dez. 2025. [28]

ROBOFEI. **RoboFEI-HT_2023_SOFTWARE**. 2023. Disponível em: https://github.com/RoboFEI/RoboFEI-HT_2023_SOFTWARE. Acesso em: 13 dez. 2025. [29]

SALEME, R. B. et al. **RoboFEI Humanoid Soccer Robot - KidSize League - Team 2025**. In: ROBOCUP 2025 SYMPOSIUM. [S.l.: s.n.], 2025. [30]

SANDERS, Mark S.; MCCORMICK, Ernest J. **Human factors in engineering and design**. 7. ed. New York: McGraw-Hill, 1993. [31]

STMICROELECTRONICS. **STM32F103x8 STM32F103xB Datasheet**: Medium-density performance line Arm®-based 32-bit MCU. Genebra: STMicroelectronics, 2015. [32]

TÉLLEZ, Ricardo; EZQUERRO, Alberto; RODRÍGUEZ, Miguel Angel. **ROS2 Basics in 5 Days**. Barcelona: The Construct Sim, 2019. [33]

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE (UFRN). Departamento de Engenharia de Computação e Automação. **Sistemas de Controle I**. Natal: UFRN, 2003. Apostila. [34]

ZANCO, Wagner da Silva. **Microcontroladores PIC: técnicas de software e hardware**. São Paulo: Érica, 2005. [35]

ZHENG, Chen et al. **Design of multi-modal feedback channel of human-robot cognitive interface for teleoperation in manufacturing.** Journal of Intelligent Manufacturing, v. 36, p. 4283–4303, 2025. Disponível em: <https://doi.org/10.1007/s10845-024-02451-x>. Acesso em: 09 dez. 2025. [36]

ANEXO A – CÁLCULO DE CONSUMO DE CORRENTE

Consumo de corrente:

$$i_{max}(ESP32 - S3) = 340mA$$

$$i_{max}(LED) = 30mA$$

$$i_{max}(Push Button) \approx 0$$

$$i_{max} = i_{max}(ESP32 - S3) + 7i_{max}(LED) = 340mA + 7 \cdot 30mA = 550mA$$

$$i_{max} = 550 \cdot 1,1 \approx 600mA$$

$$R_{en1} = \frac{V_{inicial} - V_{final}}{3\mu A}$$

$$R_{en1} = \frac{14.9V - 14.3V}{3\mu A} = 200k\Omega$$

$$R_{en2} = \frac{V_{en}}{\frac{V_{inicial} - V_{en}}{R_{en1}} + 1\mu A}$$

$$R_{en2} = \frac{1.25V}{\frac{14.9V - 1.25V}{200k\Omega} + 1\mu A} = 18.050k\Omega$$

$$R_6 = \frac{R_5 \cdot V_{ref}}{V_{out} - V_{ref}}$$

$$R_6 = \frac{10k\Omega \cdot 0.8V}{3.3V - 0.8V} = 3200\Omega$$

$$R_6 \approx 3.3k\Omega$$

$$V_{saída} = V_{ref} \cdot \left(\frac{R_5}{R_6} + 1 \right)$$

$$V_{saída} = 0.8V \cdot \left(\frac{10k\Omega}{3.3k\Omega} + 1 \right) = 3.224V$$

$$C_1 = 10\mu F \ 50V \text{ (cerâmico X5R)}$$

$$C_2 = 10\mu F \ 16V \text{ (cerâmico X5R)}$$

$$L_{\text{mínimo}} = \frac{V_{\text{saída}}(\text{max}) \cdot (V_{\text{saída}}(\text{max}) - V_{\text{saída}})}{V_{\text{entrada}}(\text{max}) \cdot K_{\text{indutor}} \cdot i_{\text{saída}} \cdot F_{\text{sw}}}$$

$$F_{\text{sw}} = \text{switching frequency} = 1\text{MHz}$$

$$L_{\text{mínimo}} = \frac{3.224\text{V} \cdot (16.8\text{V} - 3.224\text{V})}{16.8\text{V} \cdot 0.4 \cdot 300\text{mA} \cdot 1\text{MHz}} = 21,711\mu\text{H}$$

Corrente RMS do indutor: SRR6028-270Y

$$i_L(\text{rms}) = \sqrt{i_{\text{saída}}(\text{max})^2 + \frac{1}{12} \cdot \left(\frac{V_{\text{saída}} \cdot (V_{\text{entrada}}(\text{max}) - V_{\text{saída}})}{V_{\text{entrada}}(\text{max}) \cdot L_{\text{saída}} \cdot F_{\text{sw}} \cdot 0.8} \right)^2}$$

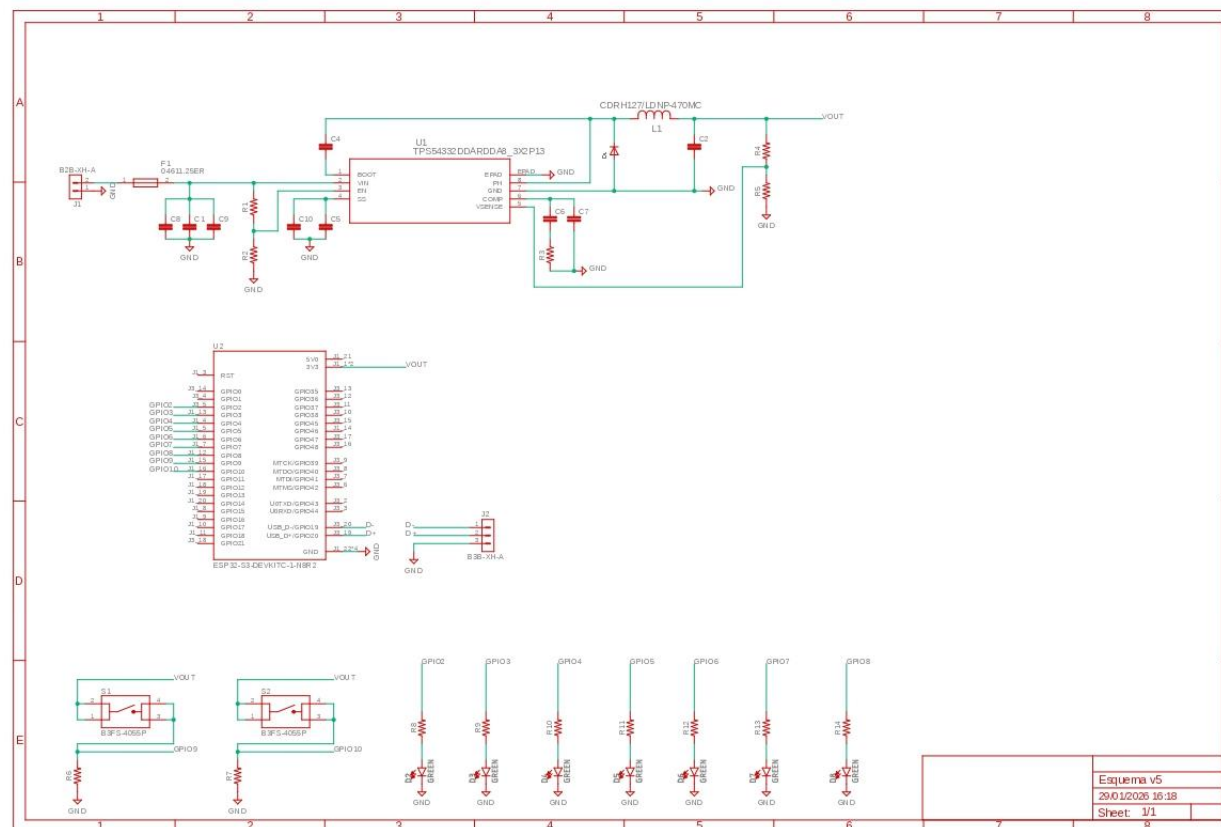
$$i_L(\text{rms}) = \sqrt{600\text{mA}^2 + \frac{1}{12} \cdot \left(\frac{3.224\text{V} \cdot (16.8\text{V} - 3.224\text{V})}{16.8\text{V} \cdot 22\mu\text{H} \cdot 1\text{MHz} \cdot 0.8} \right)^2} = 0.601\text{A}$$

Corrente de Pico do indutor: SRR6028-270Y

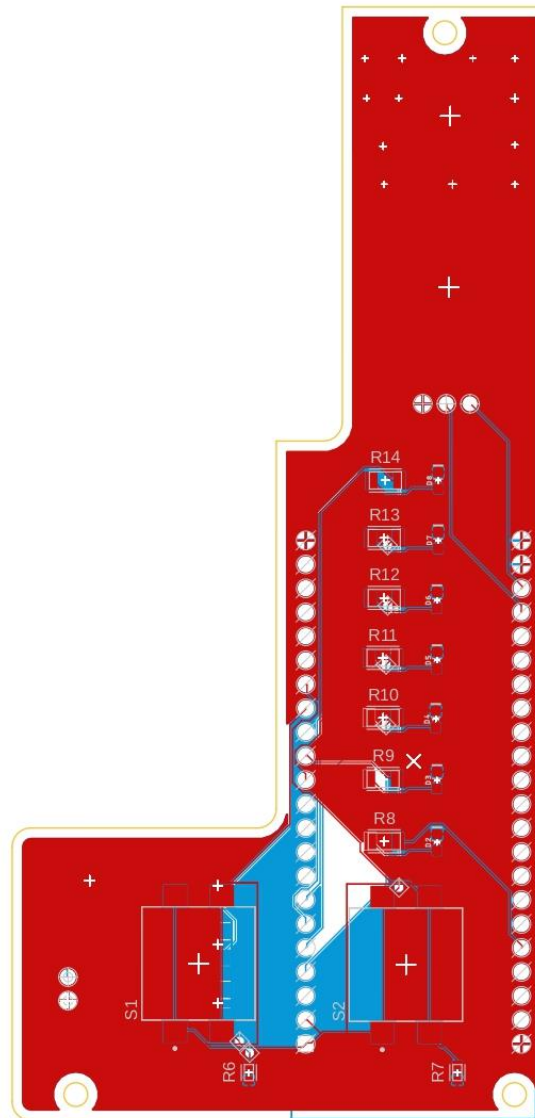
$$i_L(\text{PK}) = i_{\text{saída}}(\text{max}) + \frac{1}{12} \cdot \left(\frac{V_{\text{saída}} \cdot (V_{\text{entrada}}(\text{max}) - V_{\text{saída}})}{V_{\text{entrada}}(\text{max}) \cdot L_{\text{saída}} \cdot F_{\text{sw}} \cdot 1.6} \right)$$

$$i_L(\text{PK}) = 600\text{mA} + \frac{1}{12} \cdot \left(\frac{3.224\text{V} \cdot (16.8\text{V} - 3.224\text{V})}{16.8\text{V} \cdot 22\mu\text{H} \cdot 1\text{MHz} \cdot 1.6} \right) = 0.606\text{A}$$

ANEXO B – ESQUEMÁTICO DO PROTÓTIPO



ANEXO C – LAYOUT DO PROTÓTIPO



ANEXO D – MONTAGEM 3D DO PROTÓTIPO

